IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 28 July 2025, accepted 13 September 2025, date of publication 22 September 2025, date of current version 9 October 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3612982

SURVEY

Quantum-Resilient Cryptography: A Survey on Classical and Quantum Algorithms

H. A. SHARATH[1], JAYAVRINDA VRINDAVANAM[2], SASWATI DANA[3], AND S. N. PRASAD[4], (Senior Member, IEEE)

[1]Department of Computer Science and Engineering, Dayananda Sagar University, Bengaluru, Karnataka 562112, India
[2]Department of Computer Science and Engineering (Artificial Intelligence and Machine Learning), Dayananda Sagar University, Bengaluru, Karnataka 562112, India
[3]IBM India Research Laboratory, Bangalore 560045, India
[4]Department of Electronics and Communication Engineering, Manipal Institute of Technology Bengaluru, Manipal Academy of Higher Education, Manipal 576104, India

Corresponding author: S. N. Prasad (sn.prasad@manipal.edu)

ABSTRACT The rapid advancement of quantum computing is poised to disrupt the foundations of classical cryptography, calling into question the long-term security of widely used algorithms. Classical cryptographic techniques, including symmetric key algorithms like DES, 3DES and AES, asymmetric schemes such as RSA and ECC, and essential primitives like hash functions and the One-Time Pad (OTP), have been the cornerstone of secure digital communication since their inception. However, the progress of quantum algorithms, spearheaded by Shor and Grover, poses serious threats to these schemes, necessitating a shift in cryptographic design. This paper presents an all-encompassing review of classical cryptographic techniques, along with emerging post-quantum cryptographic (PQC) algorithms that have been developed to provide quantum-resistant security while maintaining compatibility with classical infrastructure. Concurrently, the quantum key distribution (QKD) introduces a radical shift, leveraging quantum mechanics principles to attain unconditional security. This paper elaborates both Discrete Variable (DV-QKD) and Continuous Variable (CV-QKD) approaches, highlighting their operational principles and security models. By providing a relative analysis of classical cryptographic methods, PQC, and quantum cryptography, including QKD protocols and quantum-safe primitives, this work targets to provide a consolidated insight into modern cryptographic landscape and further research toward secure communication in the quantum era.

INDEX TERMS DES, AES, RSA, Blowfish, quantum cryptography, quantum computing, post-quantum cryptography (PQC), Lattice-based cryptography, CRYSTALS-Kyber, CRYSTALS-Dilithium, code-based cryptography, McEliece, hash-based cryptography, SPHINCS+, quantum key distribution (QKD), BB84, GG02.

I. INTRODUCTION

With advancements in science and technology, modern cryptography has undergone a transformative evolution in both theory and practice. Over the past century, three major milestones have shaped its development. First, the shift from manual encryption to cipher machines significantly increased cryptographic complexity. Next, the integration of rigorous mathematical principles established cryptography as a scientific discipline, leading to the emergence of asymmetric cryptography. Most recently, driven by the demand for heightened security, quantum cryptography (QC), leveraging the laws of quantum mechanics, has been evolved.

This paper, through Sections II to VII, provides a structured survey of cryptographic algorithms and mechanisms that form the foundation of secure communication systems. Section II outlines classical cryptography. Section III focuses on core cryptographic algorithms, including symmetric encryption (e.g., AES), asymmetric schemes (e.g., RSA), the theoretically unbreakable One-Time Pad (OTP), and

The associate editor coordinating the review of this manuscript and approving it for publication was Siddhartha Bhattacharyya.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

cryptographic hash functions (e.g., SHA). Section IV explores quantum cryptographic constructs involving qubit manipulation through unitary quantum gates, Dirac notation for quantum state representation, and state visualization using the Bloch sphere. Section V investigates quantum algorithms such as Shor's integer factorization and Grover's amplitude amplification for unstructured search. Section VI surveys lattice-based and hash-based post-quantum cryptographic algorithms resistant to quantum attacks. Section VII discusses quantum key distribution (QKD), emphasizing protocols like BB84 that leverage quantum superposition and no-cloning for secure key exchange and GG02 leverages on amplitude and phase of light.

II. CRYPTOGRAPHY

Cryptography is the science and discipline of developing, analyzing, and applying mathematical techniques, algorithms, and protocols to protect information and secure communication. It involves converting information into a secret code (encryption) to prevent unauthorized access, and then converting it back (decryption) for authorized users.

Cryptography is essential for protecting sensitive information like personal data, financial transactions, and passwords. It ensures confidentiality, integrity, authentication, and non-repudiation by preventing unauthorized access, tampering, and forgery in both digital and physical systems.

Confidentiality protects data from unauthorized access; ensuring only intended recipients can view it. Integrity safeguards data from alteration, guaranteeing its accuracy and reliability. Authentication verifies the identity of communicating parties, preventing impersonation. Non-repudiation ensures that message originators cannot deny sending it, providing accountability.

In cryptography, confusion and diffusion are key principles for securing data. Confusion obfuscates the association between the ciphertext and the key, making it hard to deduce the key; this is achieved using S-boxes (Substitution boxes), which perform nonlinear substitutions. Diffusion ensures that a single plaintext bit affects many bits in the ciphertext, hiding patterns; this is done using P-boxes (Permutation boxes), which rearrange bits. Together, they make ciphers like DES and AES more secure by complicating analysis and increasing resistance to attacks. S-boxes provide confusion, while P-boxes enhance diffusion, both essential in modern block cipher designs.

III. KEY CONCEPTS OF CRYPTOGRAPHY

A. SYMMETRIC KEY CRYPTOGRAPHY

Also known as private key cryptography, it uses a pre-shared single key for both encryption and decryption. Usually it supports one-to-one secure communication. Key has been generated either by sender or by a trusted central authority (figure 1). There are several symmetric key algorithms includes AES, DES, Triple DES, Twofish, Blowfish, etc.

Sender: Cipher = Encrypt(Plain_text, key)

Receiver: Plain_text = Decrypt(Cipher, key)

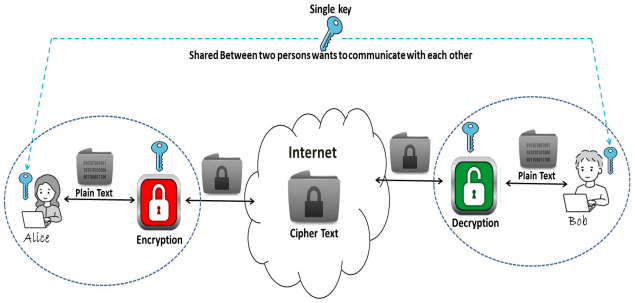


FIGURE 1. Symmetric key cryptography.

Where, Decrypt $\Longrightarrow$ inverse of Encrypt $=$ (Encrypt)$^{-1}$

Symmetric encryption ensures strong data protection and is commonly applied in areas like digital signatures and certificate validation; however, it faces challenges such as slower processing performance and complicated key distribution and management [1]. The selection of an appropriate key size constitutes a pivotal decision. A major scalability challenge arises when generating unique keys for a large user base, as the number of required keys can surge exponentially almost relative to the square of participant numbers. And the management of keys for such a vast number of individuals is equally complex [2].

1) DES-DATA ENCRYPTION STANDARD

In 1972 the Data Encryption Standard (DES) was created by IBM and formally adopted by the National Institute of Standards and Technology (NIST) originally established as the National Bureau of Standards (NBS) [3]. DES is a symmetric key algorithm that utilizes a 56-bit key on 64-bit data. Numerous rounds of encryption and decryption are accomplished by subkeys in DES. However, with advances in modern technology, DES now faces a critical vulnerability, as its small key size makes it susceptible to brute-force attacks [4], [5]. DES is a block cipher that uses a 56-bit key to secure 64-bit data chunks by first applying an initial permutation, then iterating through 16 rounds of Feistel operations involving expansion, S-box substitution, permutation, and XOR with round-specific keys derived from the main key, and finally, applying an inverse initial permutation to yield the ciphertext is shown in figure 2.

Algorithm: ***Round_Key_Generation(Parity_Bits)***
//Input: Parity Bits or Initial vector
//Output: 16 Round Keys

1: Initial_Key ← ParityDrop(Parity_Bits)
2: Split Initial_Key into $LH\text{-}K_0$ and $RH\text{-}K_0$
3: **for** $i \leftarrow 1$ to 16 **do**
4: $\quad LH\text{-}K_i \leftarrow$ left_shift($LH\text{-}K_{i-1}$)
5: $\quad RH\text{-}K_i \leftarrow$ left_shift($RH\text{-}K_{i-1}$)
6: $\quad Key_i \leftarrow$ Permutations($LH\text{-}K_i + RH\text{-}K_i$)
7: **end for**

In DES, S-boxes introduce confusion by substituting input bits with non-linear outputs, while the P-box ensures

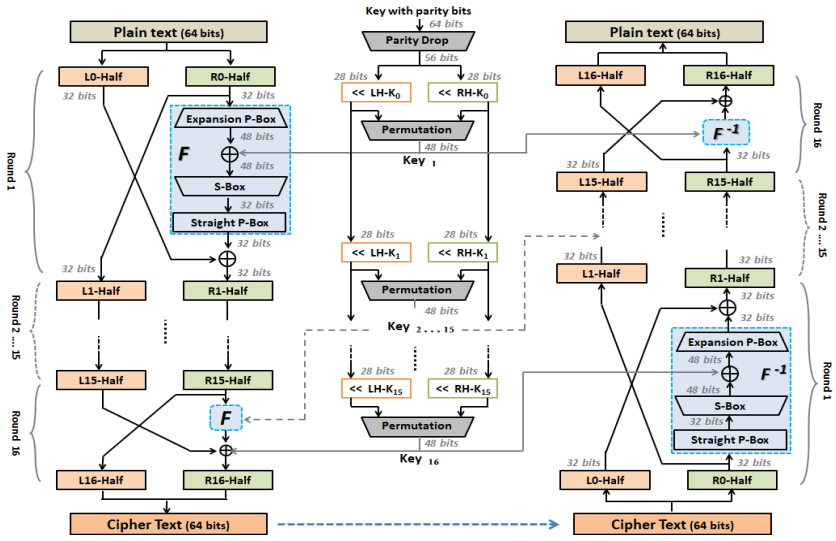


FIGURE 2. DES encryption and decryption process.

Algorithm: ***DES_Encryption(Plain_Text)***
//Input: Plain text
//Output: Cipher text

1: $LH_0 = L_{half}$ of Palin Text
2: $RH_0 = R_{half}$ of Palin Text
3: **for** $i \leftarrow 1$ to 16 **do**
4: $\quad LH_i = RH_{i-1}$
5: $\quad RH_i = LH_{i-1} \oplus F(RH_{i-1}, K_i)$
6: **end for**

Algorithm: ***DES_Decryption(Ciepher_Text)***
//Input: Cipher text
//Output: Plain text

1: $LH_0 =$ Left half of cipher Text
2: $RH_0 =$ Right half of cipher Text
3: **for** $i \leftarrow 1$ to 16 **do**
4: $\quad LH_i = RH_{i-1}$
5: $\quad RH_i = LH_{i-1} \oplus F^{-1}(RH_{i-1}, K_{n-i+1})$
6: **end for**

diffusion by permuting these outputs to spread bit influence across the block.

2) 3DES–TRIPLE DES

The Triple Data Encryption Algorithm (TDEA), commonly known as Triple DES (3DES), was initially proposed by IBM in 1998. It provides enhanced security over DES by encrypting data three times using multiple keys with increasing key length, to protect against the brute-force attack. However, it is slower and less efficient compared to the newer algorithms. Each 64-bit data block is encrypted by using either two or three 56-bit keys, depending on the configuration. The encryption process involves: encrypting with the first key, decrypting with the second key, and encrypting again with the third key. The decryption process is reverse order of encryption: decrypt with the third key, encrypt with the second key, and decrypt with the first key, shown in figure 3.

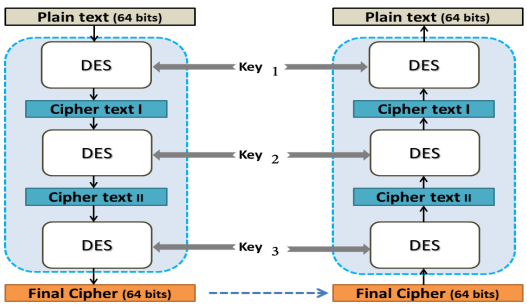


FIGURE 3. 3DES encryption and decryption process.

The encryption and decryption of the 3DES algorithm can be expressed as follows [3]:

$$Cipher_text = ENC_{K3}(DEC_{K2}(ENC_{K1}(Plain_text)))$$
$$Plain_text = DEC_{K3}(ENC_{K2}(DEC_{K1}(Cipher_text)))$$

Where, k1,k2 and k3 represents keys.

3) BLOWFISH

Blowfish is a license-free, unpatented, and freely available encryption algorithm, introduced in 1993 by cryptologists Bruce Schneier. It is fast, flexible, and highly secure. Its key lengths are flexible, supporting sizes from 32 bits up to 448 bits. It encrypts 64-bit data a block using a Feistel structure with 16 rounds, where each round includes key-dependent substitutions and permutations is shown in figure 4. The P-array is a subkey array, consists of eighteen 32-bit subkeys initially filled with hexadecimal digits.

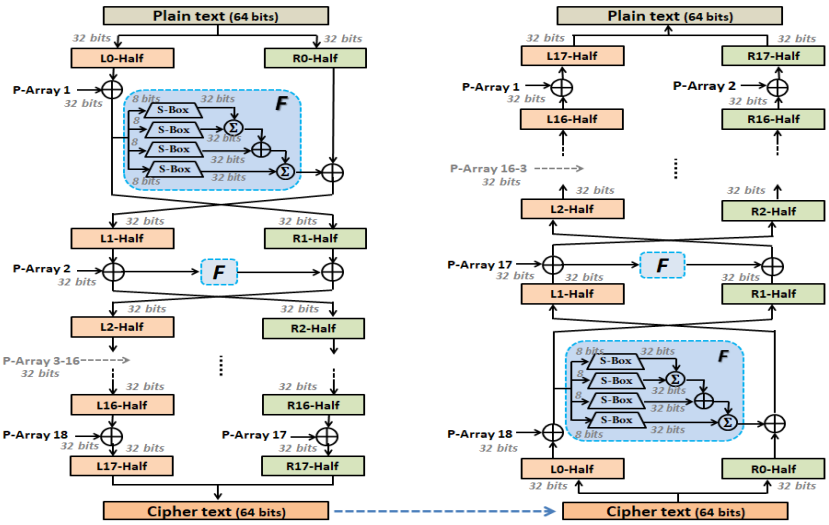


FIGURE 4. Blowfish algorithm encryption and decryption process.

Algorithm: ***Blowfish_Encryption(Plain_Text)***
//input: Data (message)
//Output: Cipher

1: $Plain_Text_{64bits} \Rightarrow L_0 - Left_{32bits} : R_0 - Right_{32bits}$
2: **for** $i \leftarrow 1$ to 16 **do**
3: $\quad R_i \leftarrow L_{i-1} \oplus P\text{-}Array_i$
4: $\quad L_i \leftarrow F(R_i) \oplus R_{i-1}$
5: **end for**
6: $R_{17} \leftarrow R_{16} \oplus P\text{-}Array_{17}$
7: $L_{17} \leftarrow L_{16} \oplus P\text{-}Array_{18}$
8: $Cipher_Text \Leftarrow L_{17} : R_{17}$
9: **return** *Cipher_Text*

Algorithm: ***Blowfish_Decryption(Cipher_Text)***
//input: Cipher
//Output: Plain text (message)

1: $Cipher_Text_{64bits} \Rightarrow L_0\text{-}Left_{32bits} : R_0\text{-}Right_{32bits}$
2: **for** $i \leftarrow 1$ to 16 **do**
3: $\quad R_i \leftarrow L_{i-1} \oplus P\text{-}Array_{18-(i-1)}$
4: $\quad L_i \leftarrow F(R_i) \oplus R_{i-1}$
5: **end for**
6: $R_{17} \leftarrow R_{16} \oplus P\text{-}Array_2$
7: $L_{17} \leftarrow L_{16} \oplus P\text{-}Array_1$
8: $Plain_Text \Leftarrow L_{17} : R_{17}$
9: **return** *Plain_Text*

To generate key-dependent P-array values, the user's key is cyclically XORed with the initial P-array values [3].

The function $F(R_i)$ is expressed as follows:

$$F(R_i) = ((SB_1[r_1] + SB_2[r_2]) \oplus SB_3[r_3]) + SB_4[r_4]$$

where, $R_{i(32bit)} \Rightarrow r_{1(8bits)}{:}r_{2(8bits)}{:}r_{3(8bits)}{:}r_{4(8bits)}$
SB_1, SB_2, SB_3, SB_4 are Substitution-boxes

In Blowfish, S-boxes provide confusion through complex, key-dependent substitutions, while the P-array functions as a P-box, permuting data to ensure diffusion.

4) AES-ADVANCE ENCRYPTION STANDARDS

AES, short for Advanced Encryption Standard, is a symmetric-key algorithm that operates as a block cipher. Originally named Rijndael, this algorithm was developed in 2001 by Vincent Rijmen and Joan Daemen. AES encryption and decryption is three times the faster than DES and is more securer than DES [4] and 3DES [3], [6]. The Advanced Encryption Standard (AES) utilizes bijective S-boxes to create an invertible algorithm with a nonlinearity measure of 112 for 8-bit balanced Boolean functions [7].

AES has several configuration parameters: key length (128, 192, or 256 bits), chaining mode (e.g., ECB, CBC etc..), and padding mode (e.g., None, Zeros, PKCS7). In encryption, the process consists of multiple rounds, each designed to enhance data security through a series of transformations is shown in figure 5. It encrypts 128-bit data blocks using 128-bit, 192-bit, or 256-bit keys, which dictate 10, 12, or 14 rounds of encryption, respectively. The first round includes only one operation, the middle rounds involve four distinct steps, and the final round uses three [5]. These steps typically include Add_Round_Key-XOR with a round key: combined data with a key using an XOR operation; Sub_Bytes-non -linear substitution: which replaces each byte using an S-Box to increase unpredictability; Shift_Rows -row wise permutation: where rows of data are circularly shifted to redistribute bytes; and Mix_Columns- column mixing using matrix multiplication: which blends bytes within columns to further obscure the data pattern [8]. Decryption is reverse of encryption, which uses inverse operations of encryption in reverse order [5]. The key schedule expands the original key into round keys is shown in figure 5.

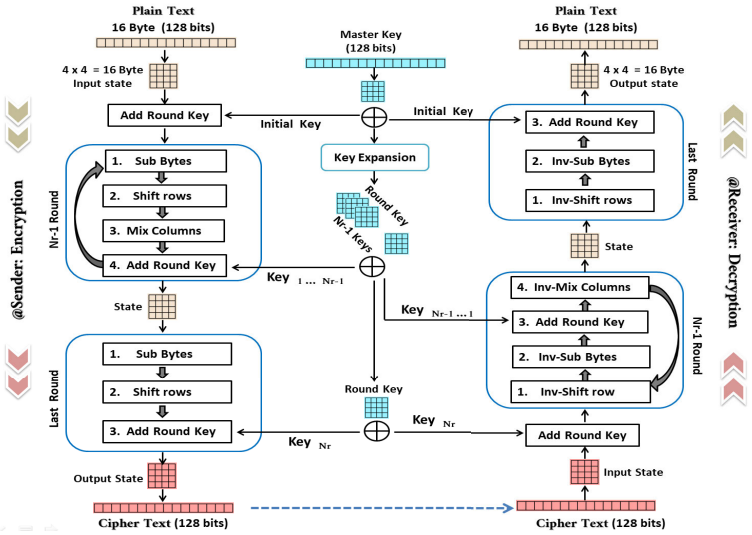


FIGURE 5. AES encryption and decryption procedure.

Matrix data formation in AES organizes the 128-bit block into a 4 × 4 byte matrix, allowing structured, byte-level operations that enhance diffusion and simplify transformations like ShiftRows and MixColumns. AES uses S-boxes in SubBytes for confusion, while ShiftRows and MixColumns act like P-boxes to achieve diffusion across the ciphertext. This algorithm is more appropriate for applications that need real-time processing and good levels of protection [4].

Algorithm: ***AES_Encryption(Plain_Text)***
//Input: Text message (Data)
//Output: Cipher text

$State_{4X4} \leftarrow$16 Byte Plain Text
AddRoundKey(*State*, Key_1)
for *round* $\leftarrow$ 1 to $Nr - 1$ **do**
  Sub-Bytes(*State*)
  Shift-Rows(*State*)
  Mix-Columns(*State*)
  AddRoundKey(*State*, Key_{round})
end for
Sub-Bytes(*State*)
Shift-Rows(*State*)
AddRoundKey(*State*, Key_N)

The study by Faheem et al., symmetric encryption algorithms were comparatively analysed —DES, 3DES, AES, and Blowfish—based on their encryption and decryption throughput. The table 1 and accompanying graph (Figure 6) with the fastest encryption and decryption speeds, Blowfish achieves the highest throughput among the tested algorithms. Conversely, 3DES shows the lowest throughput values, reflecting its computational intensity due to triple encryption. These findings highlight Blowfish's efficiency, making it a suitable choice for applications where processing speed is critical [3].

Algorithm: ***AES_Decryption(Cipher_Text)***
//Input: Cipher text
//Output: Plain text

$State_{4X4} \leftarrow$16 Byte Cipher Text
AddRoundKey(*State*, Key_N)
for *round* $\leftarrow$ *Nr* to 2 **do**
  Inv_Sub-Bytes(*State*)
  Inv_Shift-Rows(*State*)
  Inv_Mix-Columns(*State*)
  AddRoundKey(*State*, Key_{round})
end for
Inv_Sub-Bytes(*State*)
Inv_Shift-Rows(*State*)
AddRoundKey(*State*, Key_1)

TABLE 1. Throughput of symmetric key algorithms [3].

Algorithm/ Process	DES	3DES	AES	Blowfish
Encryption	4.01	3.45	4.174	25.892
Decryption	6.347	5.665	6.452	18.72

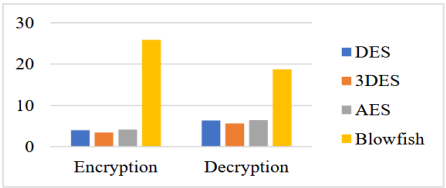


FIGURE 6. Throughput of symmetric key algorithms (Megabyte/Sec) [3].

B. ASYMMETRIC KEY CRYPTOGRAPHY

Also known as public key cryptography, it uses a uniquely co-related pair of keys (public and private) for

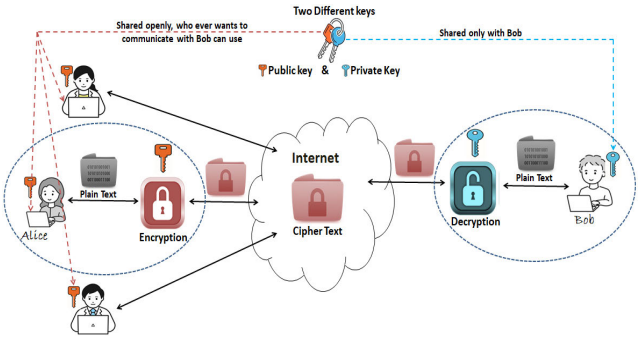


FIGURE 7. Asymmetric key cryptography.

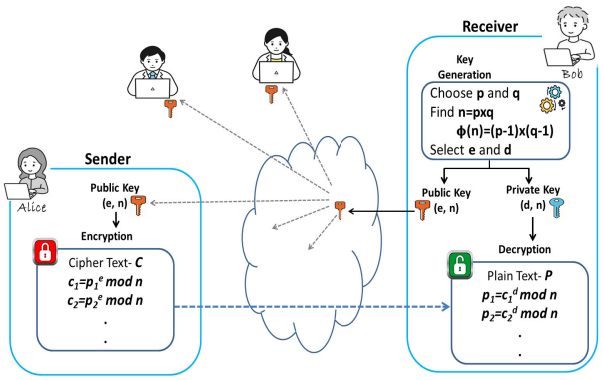


FIGURE 8. RSA encryption and decryption process.

encryption and decryption. It supports many-to-one secure communication-multiple people can encrypt using the same broadcasted public key, only owner with corresponding private key can decrypt. The sender retrieves the receiver's public key from a directory, encrypts the message, and the receiver uses their private key to decrypt it (figure 7). Both the keys are been generated either by trusted central authority or by each individual (owner/receiver end). There are several symmetric key algorithms includes RSA, Elliptic Curve Cryptography (ECC), etc.

Sender: $Cipher = Enc(Plain_text, key_{Public})$

Receiver: $Plain_text = Dec(Cipher, key_{Private})$

The private key is secret, as only the public key is transmitted in asymmetric key cryptography; it eliminates the need for a secure channel to exchange keys and reduces the risk of interception. Managing keys is also easy. But computationally expensive and requires more CPU and memory. Compared to symmetric encryption, asymmetric algorithms require much longer keys to achieve similar levels of security [9].

1) RSA-RIVEST, SHAMIR, AND ADLEMAN

RSA is public key cryptographic algorithm, It was published in 1977 and is named for its creators, Leonard Adleman, Adi Shamir, and Ron Rivest. RSA's security is fundamentally built on the computational challenge of factoring the product of two big prime figures, known as the "factoring problem" [10]. Euler's theorem is a key mathematical concept behind the RSA algorithm. Euler's theorem says that, if two integers' p and q are relatively prime, then

$$p \equiv 1 \pmod{q} \text{ or } \gcd(p, q) = 1$$

Algorithm: ***RSA_Encryption(Plain_text P)***

//Input: Plain text $P \leftarrow p_1 + p_2 + \ldots + p_n$

//Output: Cipher text $C \leftarrow c_1 + c_2 + \ldots + c_n$

1: **for** $i \leftarrow 1$ to n **do**
2: $\quad c_i \leftarrow p_i^e \bmod n$
3: **end for**

Figure 8 shows the RSA algorithm in three steps. First, two primes p and q are chosen to compute n and $\phi(n)$ The public key e, n and private key d, n are generated using e and d. Next, encryption turns message M into ciphertext $C = M^e \bmod n$. Finally, decryption recovers M using $C^d \bmod n$.

Euler's totient function $\phi(n)$ counts how many positive integers less than n are coprime to n. For a prime number n, $\phi(n)$ is always $n-1$ since every integer less than n is relatively prime to it. If p and q are relatively prime, then $\phi(pq) = \phi(p) \times \phi(q)$. So, if a number is formed by multiplying two prime numbers, p and n, then $\phi(pq) = \phi(p) \times \phi(q) = (p\text{-}1) \times (q\text{-}1)$ [11], [12].

Algorithm: ***RSA_Decryption(Cipher_text C)***

//Input: Cipher text $C \leftarrow c_1 + c_2 + \ldots + c_n$

//Output: Plain text $P \leftarrow p_1 + p_2 + \ldots + p_n$

1: **for** $i \leftarrow 1$ to n **do**
2: $\quad p_i \leftarrow c_i^d \bmod n$
3: **end for**

Algorithm: ***RSA_KeyGen( )***

//Output: Public key and Private key

1: Choose two prime numbers, p and q, $p \neq q$
2: *Calculate* : $n \leftarrow p$ x q
$\quad \phi(n) \leftarrow (p-1)\text{x}(q-1)$
3: **for** $1 < e < \phi(n)$ and $gcd(\phi(n), e) = 1$ **do**
4: $\quad KEY_{public} \leftarrow (e, n)$
5: $\quad$ **for** $1 < d < \phi(n)$ and (d x e mod ϕ(n)) = 1) **do**
6: $\quad\quad KEY_{private} \leftarrow (d, n)$
7: $\quad$ **end for**
8: **end for**
9: **return** KEY_{public} and $KEY_{private}$

2) ECC–ELLIPTIC CURVE CRYPTOGRAPHY

Elliptic Curve Cryptography (ECC) is a public key cryptographic technique that uses the elliptic curve's mathematics over finite fields to offer secure and efficient encryption. ECC's cryptographic strength stems from the intractability of the Elliptic Curve Discrete Logarithm Problem (ECDLP)-although computing multiples of a point through scalar multiplication is straightforward and easy, reversing the

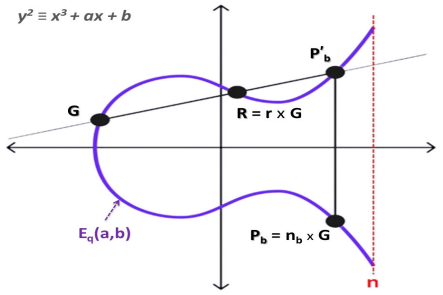


FIGURE 9. Secured hashing Algorithm SHA-256.

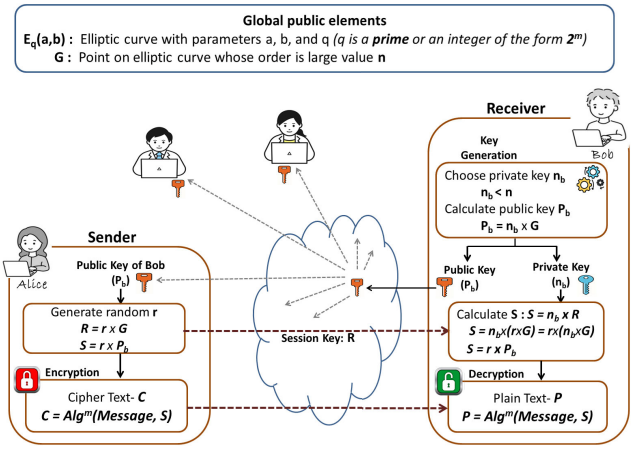


FIGURE 10. ECC encryption and decryption process.

process to find the original scalar is extremely difficult—this asymmetry forms the core of ECC's security. Cryptographic processes like digital signatures, encryption, and key generation in ECC use points on an elliptic curve over finite fields F_p (where p is a prime) defined by an equation:

$$y^2 = x^3 + ax + b \pmod{p}$$

Figure 10 shows the ECC algorithm; a base point G on the elliptic curve is selected, as shown in Figure 9. The private key n_b is multiplied by G to get the public key $P_b = n_b\ G$. For encryption, the sender chooses a random integer r and computes the ciphertext as two points: $R = r\ G$ and $S = r\ P_b$. For decryption, the receiver uses their private key S to recover the original message.

ECC provides analogous security to RSA with significantly smaller keys. For instance, a 256-bit key in ECC provides a level of security comparable to a 3072-bit RSA key [13], [14].

The selection of the appropriate parameters of the elliptic curve is essential to ensure security. Inadequately chosen curves can expose the system to vulnerabilities or significantly weaken its security.

C. HASH FUNCTIONS

Hash functions creates a fixed-size digital fingerprint of given data (figure 11). A hash function is a critical component of information security, serving a vital role in contemporary cryptographic systems by converting input

Algorithm: ***ECC_Encryption(public-key P_b, Message M)***
//Input: Receiver's public key $P_b = n_b \cdot G$, message M
//Output: Cipher text C

1: Generate an arbitrary number randomly $r \in [1, n\text{-}1]$
2: Compute ephemeral public key R:
$R = r \cdot G$
3: Compute Session key S using P_b:
$S = r \cdot P_b$
4: Encrypt the data using any asymmetric key algorithms like AES with key S
$C = ENC(M, S)$
5: Send Cipher text C

Algorithm: ECC_Decryption(private-key n_b, Ephemeral-key R, Cipher C)
//Inputs: Receiver's private key $P_b = n_b \cdot G$, message M
//Output: Plain text P

1: Compute shared Session key S using R:
$S = n_b \cdot R$
2: Decrypt the Cipher using with key S using same asymmetric key algorithms used in Encryption
$P = DEC(C, S)$

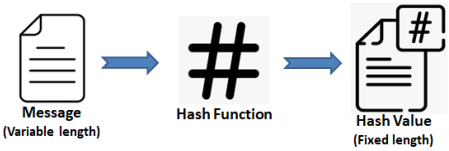


FIGURE 11. General hashing process.

data into a fixed-size string known as a hash value. This hash value uniquely represents the original input, enabling data integrity verification without revealing the actual content i.e., Irreversible [15]. It is used for password protection, authenticity of sender and data. A good Hash algorithm should be resistance to collision attack, pre-image attack and second pre-image attack [16]. Generally, hash functions are categorize in two ways: for data integrity, unkeyed hash functions are used; for message authentication, keyed hash functions is used [15].

Examples: SHA-1, SHA-256, MD5

$$Hash_value = Hash(Message)$$

1) SHA–SECURED HASH ALGORITHM

NIST has released many variants of SHA algorithm with different mathematical functions. Both SHA-0 and SHA-1 uses design principle based on Merkle Damgard's structure as MD5. In 1995 NIST introduced SHA-1 hash algorithm, it uses a series of 80 logical functions, denoted f_0 through f_{79}. The function $f_t(x, y, z)$, defined for 0<t<79, takes three 32-bit words (x, y, z) and outputs a single 32-bit word. The function

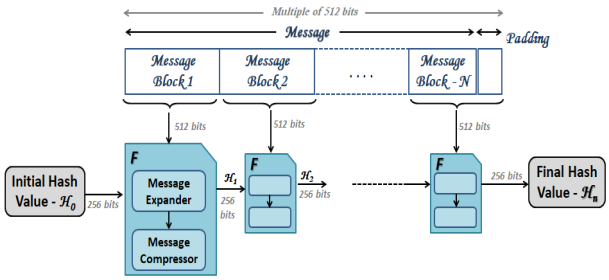


FIGURE 12. Secured hashing Algorithm SHA-256.

$f_t(x, y, z)$ is specified as follows [17]:

$$f_t(x,y,z) = \begin{cases} (x \wedge y) \oplus (\sim x \wedge z) & 0 < t < 19 \\ (x \oplus y \oplus z & 20 < t < 39 \\ (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) & 40 < t < 59 \\ x \wedge y \wedge z & 60 < t < 79 \end{cases}$$

SHA-0 and SHA-1 use compression functions derived from MD4 and MD5, making both susceptible to collision attacks. In 2002 NIST standardise SHA-2 series, which includes hash digests of 224, 256, 384, and 512 bits. The integration of nonlinear functions in compression module made SHA-2 algorithm is more robust compared to SHA-1 [16].

The figure 12 shows SHA-256 taking input data, processing it in blocks with bitwise operations input message pre-processing, block division, and iterative compression, producing a fixed 256-bit hash output that ensures secure and consistent data integrity.

Algorithm: ***SHA-2_256(Message M)***
// Input: Message M
// Output: Message digest (Hash Value) of 256 bits
// $F(m_i)$ is compression function, H_i is 256 bits-Hash value of message Block m_i

1: Message padded to a 512-bit multiple
2: Message is segmented into 512 bits blocks:
$M \Rightarrow m_1, m_2, m_3, \ldots, m_n$
3: Initial hash value: $H_0 \leftarrow$ 256 bits Initial Vector
4: **for** $i \leftarrow 1$ to n **do**
5: $H_i = H_{i-1} + F(m_i)H_{i-1}$
6: **end for**
7: **return** H_n

After a successful collision attack on SHA-1, in 2015, though SHA-1 and SHA-2 change in implementation, they are rooted in equivalent mathematical foundations, which inherently carry comparable structural vulnerabilities. Similarly, it is foreseeable that current hash functions will become increasingly susceptible to cryptographic attacks over time. In response, the U.S. federal agency NIST recognises SHA-3 (SHAKE-Secure Hash Algorithm KECCAK Extendable-output) algorithm as a standard hash function after winning a competition by Keccak on October 2, 2012. SHA-3 is more resilient hashing standard includes various versions with hash digest of 224, 256, 384, and 512 bits, with block size of 1152, 1088, 832, and 576 bits correspondingly. It is different from the SHA family, providing a secure alternative for applications requiring enhanced protection. Although existing hashing algorithms are faster in terms of software, SHA-3 is more robust and faster in terms of hardware. In near future it will replace all remaining variants of SHA algorithms standards [16].

2) TWO FACTOR AUTHENTICATION (2FA): OTP

Over the past 30 years, research has shown that password-based single-factor authentication is vulnerable to threats like spyware, interception, man-in-middle, and brute force attacks. As a more secure alternative, two-factor authentication (2FA) has become popular by combining two of three elements: knowledge, possession, or identity; ensuring protection even if one factor is compromised [18].

One of the most generally used methods for implementing two-factor authentication (2FA) is through software-based One-Time Passwords (OTP), which relies on OTP algorithms to enhance system security. One-Time Passwords (OTPs) are typically short, between 4 and 8 characters long. Service providers forwarding OTPs for users makes the length of these codes a concern [19].

Simple hash function protocol: L Lamport suggested the first OTP the S/KEY authentication method, to authenticate untrusted computers in public use. This method utilizes chain of hash functions to generate OTPs [19]. After this, Leslie Lamport also developed a version Lamport's Hash Chain Protocol that employs bounded hash chains, building upon the basic hash chain concept. Yeh, Shen and Hwang proposed YSH Protocol based on Lamport's Hash chain protocol, which uses secret key and a random number to generate OTPs. Bicakci Protocol: Asymmetric cryptography is used to generate and verify OTPs. Chefranov's Scheme is a protocol built around a sophisticated and highly secure algorithm, distinguished by its multiple configurable parameters [20].

HOTP–HMAC based One-Time Password is created in 2005, the event-based OTP algorithm leverages Hash-based Message Authentication Codes (HMAC) to generate its values. Using SHA-2/ SHA-3, HMAC generate hash value. It works by incrementing the counter value C (separate counters on both client and server side) and a static symmetric key-K (Shared among client and server) token and validation service-specific. HOTP authenticators never expire; they remain valid indefinitely, which increases their vulnerability to attacks [18].

TOTP–Time based OTP is a time-driven variant of the HOTP algorithm. The key difference is that HOTP uses a counter, while TOTP is timer based on the current time T to generate OTP. i.e., TOTP = HOTP(K, T). The standard format is YYYY-MM-DD for the date, followed by HH:MM:SS for the time. This OTP has to be validating before specified time otherwise it expires. Three factors: Randomness, Expiration(time limit) and Consumption(Attempts) makes TOTP more robust than HOTP [18].

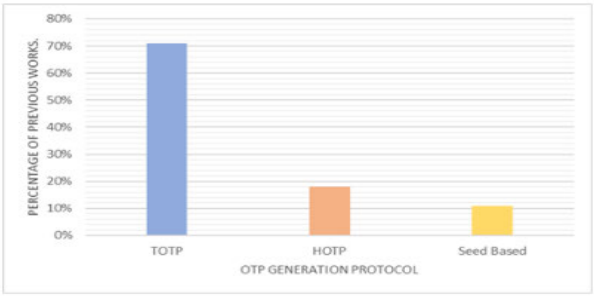


FIGURE 13. Usage of OTP protocol [18].

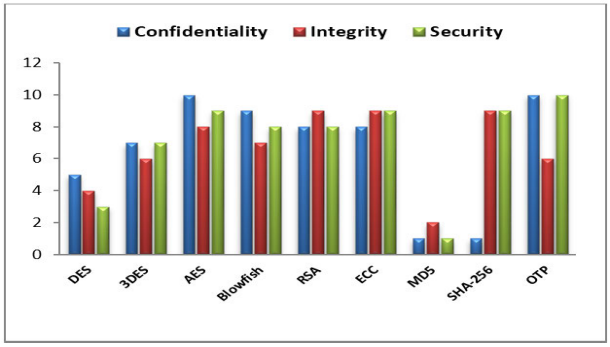


FIGURE 14. Comparison of classical cryptographic algorithms on 10 points scale [21].

TABLE 2. An assessment of the security, integrity, and confidentiality of classical cryptographic algorithms [21].

Algorithm	Confidentiality	Integrity	Security
DES	5	4	3
3DES	7	6	7
AES	10	8	9
Blowfish	9	7	8
RSA	8	9	8
ECC	8	9	9
MD5	1	2	1
SHA-256	1	9	9
OTP	10	6	10

A seed value-based OTP is generated by a Generation System (GS), a unique sequence of one-time password (OTP) values is generated by repeatedly applying a secure hash function, iterated N times by initializing with server seed value S and user provided N and password phrase PP. i.e., $OTP = Hashfuntion(PP, S)^N$ [18].

According to figure 13, in the research papers, a clear trend is evident in OTP generation protocols, with TOTP being the most widely implemented at 71%, followed by HOTP at 18%, and value-based OTP methods comprising the remaining 11% [18].

The comparative evaluation of cryptographic algorithms in Table 2 of the paper by Ramakrishna and Ali Shaik assigns weighted scores (on a scale from 1 to 10) for confidentiality, integrity, and overall security, highlighting their practical strengths and weaknesses based on their literature analysis. AES and RSA stand out for their strong performance across all three criteria, with AES offering fast, secure encryption and RSA excelling in secure key exchange and digital signatures. ECC also performs well, particularly in resource-limited environments due to its efficiency with smaller keys. On the other hand, DES scores low due to its compact key and vulnerability to brute-force attacks, while 3DES offers only moderate improvements. Blowfish provides a good balance of speed and security but is limited by its small block size. SHA-256 ensures strong integrity and security as a hash function but lacks confidentiality. OTP achieves ideal security and confidentiality in theory, though practical challenges limit its widespread use (figure 14).

Table 2 provides a comparative overview of classical cryptographic algorithms, highlighting their foundational concepts, operational mechanisms, strengths, and known vulnerabilities. This summary serves as a reference for understanding the evolution and security implications of widely used encryption and hashing techniques.

Factoring a n-bit number with classical methods is computationally expensive, typically requiring exponential time, whereas Shor's algorithm can achieve the same task in polynomial time to find the keys. Consequently, a fully developed quantum computer running Shor's algorithm would potentially compromise established public-key encryption systems like RSA and ECC, rendering them ineffective.

Grover's algorithm impact on symmetric key algorithms such as AES and SHA-256 by reducing brute force search space and simplifies the process of finding key. Though it is not dangerous, but with quantum computer, it weakens the symmetric key encryption method [22].

Advancements in quantum technology would expose to increased risk of technological surprise and strategic blind spots. The progress in quantum computing is surpassing initial predictions. Both asymmetric and symmetric key classical cryptographic algorithms are particularly at risk from quantum attacks leveraging Grover's and Shor's algorithms. The emergence of these threats necessitates the development and exploration of post-quantum cryptographic (PQC) methodologies and the critical need of developing quantum resistant substitutes.

IV. QUANTUM CRYPTOGRAPHY

Quantum cryptography utilizes ideologies of quantum mechanics to strengthen security through various cryptographic techniques. It is an intersection of Physics, Quantum mechanics, Computer Science and Mathematics. Over the past few decades, the relationship between computer science and physics has attracted growing interest from researchers. Breakthroughs in qubit stability, error correction, and quantum software are significantly accelerating progress beyond expectations.

Despite its promise of unbreakable security, quantum cryptography faces several critical challenges. High implementation costs, the need for dedicated hardware, and scalability limitations—especially over extensive distances—pose significant barriers. Moreover, vulnerabilities in quantum systems, such as side-channel attacks and

TABLE 3. Overview of classical cryptographic algorithms.

Algorithm	Year Invented	Mathematical Concept	Basic Operations	Strength	Possible Attacks
DES	1975	Feistel network, 56-bit key	16 rounds of substitution and permutation	Weak by today's standards	Brute force, Differential cryptanalysis
3DES	1998 (standardized)	Repeated DES (Encrypt-Decrypt-Encrypt)	Applies DES 3 times with 2 or 3 keys	Stronger than DES, but slow	Meet-in-the-middle attack
AES	2001	Substitution-permutation network (SPN), Rijndael	SubBytes, ShiftRows, MixColumns, AddRoundKey	Strong, efficient, supports 128/192/256-bit keys	Side-channel, related-key attacks
Blowfish	1993	Feistel network, variable key (32–448 bits)	16 rounds of Feistel cipher	Strong, fast, but block size is small (64-bit)	Birthday attacks, key exhaustion (small blocks)
RSA	1977	Integer factorization of large primes	Modular exponentiation, public/private key pair	Strong if large key used (>2048 bits)	Timing attacks, chosen ciphertext
ECC	1985	Elliptic curve algebra over finite fields	Point multiplication, modular arithmetic	Strong with shorter keys (e.g., 256-bit ECC $\approx$ 3072-bit RSA)	Invalid curve attacks, side-channel
SHA	SHA1 - 1995, SHA2 - 2001, SHA3 - 2015	One-way hash functions, Merkle–Damgard and sponge constructions	Compression functions, bitwise operations	Strong (SHA-2/3), SHA-1 is broken	Collision attacks (SHA-1), length extension (SHA-2)
OTP	1882	Modular addition with truly random key	Bitwise XOR with key of same length as message	Perfect security (provable)	None if implemented correctly

photon-number-splitting, can compromise security in real-world deployments. Integration with classical infrastructure is also difficult due to lack of standardization and interoperability. Regulatory gaps and the absence of unified global standards further complicate its adoption, requiring collaborative international efforts to ensure secure, scalable, and inclusive deployment of quantum cryptographic systems [23].

A. FUNDAMENTALS OF QUANTUM COMPUTING

Quantum computing is a relatively latest field that emerged in the late 1970s, initially as a concept in science fiction. Richard Feynman introduced the idea of a quantum computer in 1981. This sparked broader interest in the field during the early 1980s. In 1985, David Deutsch well defined the concept of a quantum computer, which marked a significant advancement in quantum computing. In the subsequent years, key advances highlighted quantum computing's potential to outperform classical systems. Notably, Simon and Shor developed algorithms showing major speedups in tasks like prime factorization and cryptography [24].

Notwithstanding its many real-world drawbacks, quantum computing has enormous potential and a wide range of applications across diverse fields. Even as quantum computing is just beginning to evolve, its potential to revolutionize traditional computing infrastructure makes it increasingly important [25]. Figure 15 shows the three decade evolution of QC. QC's applications span secure communication, authentication, healthcare, financial services, data centres, IoT security, and secure voting systems. Over the past three decades, quantum computing has progressed from theoretical foundations to experimental breakthroughs and toward practical applications. In the 1990s, the development of algorithms like Shor's and Grover's highlighted the promise of quantum advantage. The 2000s saw early physical implementations of qubits and secure communication via quantum key distribution. During the 2010s, improvements in engineering made tiny quantum computers possible, and people could access them over the internet using cloud services. In the 2020s, the field entered a new phase with showcase of quantum supremacy, the advent of hybrid quantum-classical systems, and increasing focus on scalable, fault-tolerant architectures, positioning quantum computing as a ground-breaking technology across industries [26].

1) QUANTUM COMPUTING PRINCIPLES FOUNDATION FOR QUANTUM CRYPTOGRAPHIC

Quantum computing, built on principles like superposition, entanglement, and interference, enables new ways of processing information beyond classical limits. These principles

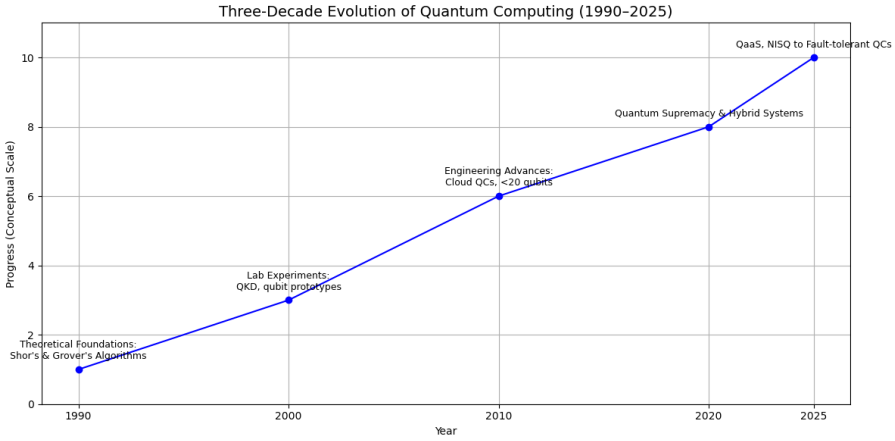


FIGURE 15. Three-decade evolution of quantum computing (1990-2025).

form the foundation of quantum cryptography, delivering quantum-safe communication solutions, such as distribution of quantum keys, offering security against both classical and quantum attacks.

The *Entanglement principle* (or quantum entanglement), It's a core principle of quantum mechanics; perform indispensable part in the progress of quantum computing and cryptography by offering superior capabilities that surpass classical cryptographic methods. Entanglement often termed by Einstein as ''spooky action at a distance'': permits qubits to be correlated such that the state of one qubit influences the state of another, even over long distances. Entanglement is at the heart of quantum physics and future quantum technologies. Although entanglement offers advantages in quantum cryptography, substantial challenges persist in scaling these systems for long-distance communication and integrating them into existing networks [24], [25].

The *Superposition* is a core concept which enables a quantum system to be in various states at the same time. Each qubit can represent a combination of two states simultaneously, and therefore, with n qubits, a quantum system can explore all 2^n potential states at once due to superposition. This allows quantum computers to process all these states in parallel, potentially solving problems in a single operation that would require 2^n steps on a classical computer [24], [25].

The *Heisenberg Uncertainty principle* is a cornerstone of quantum mechanics, formulated by Werner Heisenberg in 1927. It states that certain pairs of physical properties are complementary, meaning they are interrelated in such a way that measuring one necessarily disturbs the other, making it impossible to measure them individually without mutual influence. The uncertainty principle is not just a limitation but also a condition for reconciling mutually exclusive experimental options [27], [28].

The *No-Cloning principle*(or no-cloning theorem), was officially proven back in 1982 by William Wootters and Wojciech Zurek, with Dennis Dieks coming to the same conclusion independently. This is a consequence of the linear structure inherent in quantum mechanical systems that says you can't make an exact replica of an unidentified quantum state. This limitation is actually really useful in quantum cryptography, because it means any attempt to eavesdrop on quantum communication can be caught pretty easily [25], [29].

Coherence refers to how long a quantum system (like a qubit) can sustain its quantum properties—especially entanglement and superposition. The loss of coherence is called decoherence: transforms a pure quantum state into a mixed quantum state, destroying quantum effects when it exposed to environment. When qubit interacts with the environmental conditions like thermal noise, electromagnetic fields, or measurement errors, it starts behaving like a classical bit — collapsing into either 0 or 1. *Decoherence* makes quantum information, stored in the off-diagonal parts of a density matrix, decay over time [25].

Interference in quantum computing is a fundamental concept that allows quantum algorithms to enhance correct answers and cancel out wrong ones—by combining quantum amplitudes constructively or destructively; Constructive interference boosts the probability of the correct answer. Destructive interference reduces or cancels out incorrect paths. It works similar to how wave interference works in physics.

B. CLASSICAL BITS V/S QUANTUM BITS

A classical bit in short bit (binary digit) is the fundamental unit of information in classical cryptography. It can take only one of two possible values: 0 or 1, typically represented by physical states like electrical voltage, light pulses or magnetic polarity. A table 4 comparing bits and qubits. Bits are binary and deterministic — no ambiguity or in-between states. Total n bits can represent 2^n different characters. In contrast, a Quantum bit in short qubit is used to represent and transmit quantum information, primarily for secure communication. Quantum Cryptography relies on Qubits,

TABLE 4. Comparison of classical Bit and quantum Qubit.

Bit	Qubit
It takes the value either 0 or 1	It is superposition of 0 and 1 simultaneously
Binary and deterministic	Linear and probabilistic
Relay on properties of Classical Mechanics	Relay on properties of Quantum Mechanics
Boolean algebra based	Linear algebra based
Can perform logical operations using logical gates and logical circuits	Can perform Quantum operation using quantum gates and quantum circuits
Bit operations are non-reversible	Qubit operations are reversible
Series of photons (usually $10^6 - 10^9$) are used to represent single bit for long distance communication through optical fibre	Single photon is used to represent single qubit for long distance communication through dark fibre

analogous to the binary digits it can exist in a superposition of 0 and 1 simultaneously; it can assume either a 0 or a 1 value. A qubit can be in a combination of both 0 and 1 states. A set of qubits can represent exponentially more values than same number of classical-bits. In general, n qubits possess values of 2^n classical bits [30]. Qubits are linear and probabilistic — in-between states. Since, quantum bits can exist in multiple states simultaneously, allowing quantum computers to process a vast number of possibilities in parallel. Qubits are realized using quantum systems like electron spins, photon polarizations, Ion trap states, or superconducting circuits, and are often visualized on a Bloch Sphere to represent their complex state space [24], [29].

Qubits can be created using various technologies, each with unique strengths and limitations. Superconducting qubits, formed using two-level superconducting circuits, offer fast operations with decent gate fidelity (99.4%) but face challenges like short lifetimes (50–100 μs), low coupling rates, and the need for extremely low operating temperatures. Ion trap qubits, which use the spin states of charged ions, have exceptional coherence times (>1000 s) and very high fidelity (99.9%), yet suffer from slow speeds and complex initialization and readout, which limit scalability. Nuclear Magnetic Resonance (NMR) qubits utilize nuclear spins within molecules and share similar lifetimes with ion traps, though they lag in fidelity (95%) and struggle with signal-to-noise ratios and weak entanglement. Spin Transfer Torque-based qubits rely on nuclear or electronic donor spins and show promise for high scalability with respectable fidelity (99%), but are hindered by weak spin-orbit coupling, long spin relaxation times, and energy dissipation. Semiconductor spin qubits, built by placing electrons in pure silicon and controlling them via microwaves, also achieve 99% fidelity and high scalability but face difficulties in gate precision, donor placement accuracy, and background disorder. Each technology involves trade-offs between fidelity, coherence, scalability, and engineering complexity, shaping their suitability for practical quantum systems [24].

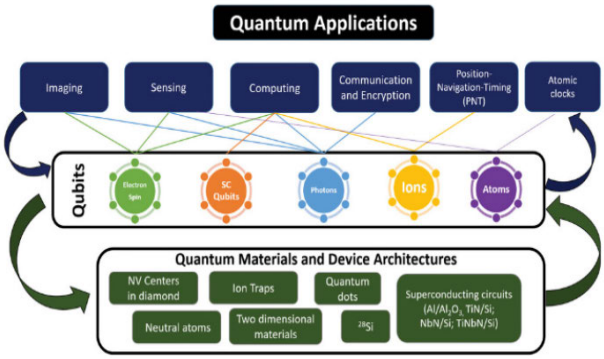


FIGURE 16. Association of quantum bit types with specific tasks [31].

The figure 16 illustrates how quantum materials give rise to various qubit types, each suited to specific tasks like computing, sensing, and communication. The choice of qubit depends on its physical properties and application needs. It highlights the flow from materials to impactful quantum technologies [31].

C. QUANTUM STATES

A qubit is defined by its quantum state. A quantum state is a mathematical representation of qubit's condition. In other words, a quantum state is a complete description of a qubit's behaviour and properties, which includes [32]:

- Pure states: basis states like $|0\rangle$, $|1\rangle$, or superposition state like $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$
- Mixed states: probabilistic combinations of pure states.

D. DIRAC'S NOTATION (⟨ Bra | and | Ket ⟩): Mathematical representation of qubit

A Hilbert space is a complex vector space equipped with an inner product, providing the mathematical framework in which quantum states are represented, manipulated, and measured. It enables the formal description of quantum superposition, measurement outcomes, and the evolution of quantum systems. Hilbert space of n dimension is representing by $\mathcal{H}_n$ [32]. The state of a qubit is described as a vector and is represented by a single column matrix on Hilbert space. A single qubit lives in a 2-dimensional Hilbert space $\mathcal{H}_2$ and basis states are mathematically represented by Dirac's bra-ket ($\langle bra|$ and $|ket\rangle$) notations with row and column matrix respectively.

$$\langle 0| = \begin{bmatrix} 0 & 1 \end{bmatrix}, \langle 1| = \begin{bmatrix} 1 & 0 \end{bmatrix}$$
$$|0\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}, |1\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

and superposition state $|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ is represented by

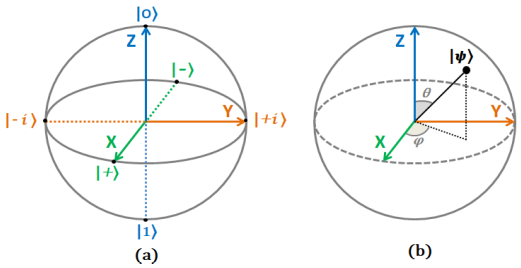


FIGURE 17. Bloch sphere representing pure states (a). Basis state (b). Superposition state.

$|\psi\rangle = \begin{bmatrix}\alpha\\ \beta\end{bmatrix}$ where, α and β are complex numbers denoted probability amplitude of state $|0\rangle$ and $|1\rangle$ respectively, with $\alpha^2 + \beta^2 = 1$.

Two qubits form a 4-dimentional Hilbert space, denoted by $\mathcal{H}_2 \oplus \mathcal{H}_2 = \mathcal{H}_4$, basis states (tensor product of individual qubits) are represented by bra-ket notations

$$\langle 00| = \begin{bmatrix}1 & 0 & 0 & 0\end{bmatrix}, \langle 01| = \begin{bmatrix}0 & 1 & 0 & 0\end{bmatrix},$$
$$\langle 10| = \begin{bmatrix}0 & 0 & 1 & 0\end{bmatrix}, \langle 11| = \begin{bmatrix}0 & 0 & 0 & 1\end{bmatrix}$$
$$|00\rangle = \begin{bmatrix}1\\0\\0\\0\end{bmatrix}, |01\rangle = \begin{bmatrix}0\\1\\0\\0\end{bmatrix}, |10\rangle = \begin{bmatrix}0\\0\\1\\0\end{bmatrix}, |11\rangle = \begin{bmatrix}0\\0\\0\\1\end{bmatrix}$$

and general state $|\psi\rangle = \alpha|00\rangle + \beta|01\rangle + \gamma|10\rangle + \delta|11\rangle$ is represented by

$|\psi\rangle = \begin{bmatrix}\alpha\\ \beta\\ \gamma\\ \delta\end{bmatrix}$, where, α, β, γ and δ are complex numbers denoted probability amplitude of state $|00\rangle$, $|01\rangle$, $|10\rangle$ and $|11\rangle$ respectively, with $\alpha^2 + \beta^2 + \gamma^2 + \delta^2 = 1$

The probability amplitudes of all possible states in a qubit are interrelated, and when the qubit is measured, its state collapses to one of the potential states.

E. BLOTCH SPHERE: VISUAL REPRESENTATION OF QUBIT

The Bloch sphere is a fundamental visualization tool in quantum computing and quantum mechanics. It illuminates a visual geometric model of the state of a single qubit within the Hilbert space. It shows all the pure quantum states a qubit can occupy as points on the surface of a sphere shown in figure 17.

The two basis states $|0\rangle$ and $|1\rangle$ are represented on the Bloch sphere as the north (positive Z-axis) and south (negative Z-axis) poles, respectively. Every other pure qubit state can be visualized as a point on the surface of the sphere, formed by a superposition of these basis states. A general qubit state $|\psi\rangle$ on Bloch sphere can be representing using two angles shown in figure 17: the polar angle $\theta \in [0, \pi]$ - angle between the state vector $|\psi\rangle$ and the Z-axis, the azimuthal angle $\varphi \in [0, 2\pi]$ - the angle between the X-axis and the shadow (projection) of the qubit's state on the XY-plane [32].

$|\psi\rangle = \cos(\frac{\theta}{2})|0\rangle + e^{-i\varphi}\sin(\frac{\theta}{2})|1\rangle$

Other important states with equal superposition of $|0\rangle$ and $|1\rangle$ but different relative phase:

$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$ - it lies on positive X-axis with no relative phase

$|-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$ - it lies on negative X-axis with relative phase of π (180 degrees)

$|+i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i|1\rangle)$ - it lies on positive Y-axis with relative phase of i (or $e^{i(\frac{\pi}{2})}$)

$|-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i|1\rangle)$ - it lies on negative Y-axis with relative phase of $-i$ (or $e^{-i(\frac{\pi}{2})}$)

F. QUANTUM GATES

Boolean algebra provides a formal and systematic framework for manipulating classical bits, forming the foundation for implementing hardware through logic gates like AND, OR, and NOT. Similarly, in quantum computing, quantum operations are driven by the fundamentals of linear algebra and unitary operations, with quantum gates—like Hadamard, Pauli-X, and CNOT—serving as the building blocks for manipulating qubits. These gates operate on quantum states, enabling superposition, entanglement, and interference, which are essential for quantum computation. Each gate operations are represented mathematically by unitary matrices shown in table 5.

Quantum gates take a quantum state as the input and output another quantum state. In order to extract/infer any information from this state, it is necessary to measure it. Quantum measurement is a nonlinear process that takes a quantum state as the input and gives some classical information (bits) as output. In its simplest form, quantum measurement is characterized with the projection operators based on some observable [24], [25], [33].

V. QUANTUM ALGORITHMS

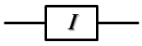
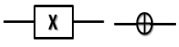
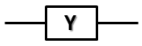
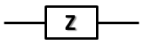
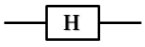
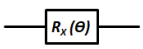
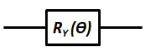
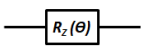
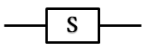
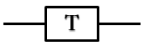
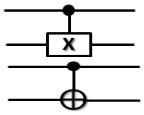
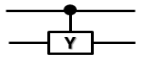
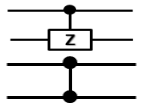
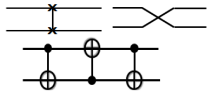
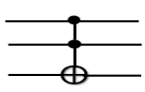
The quantum algorithm works by applying a series of quantum operations (quantum gates) to the input state. As quantum hardware advances, understanding quantum algorithms is increasingly vital for solving complex problems in cryptography, optimization, and related domains. Shor's and Grover's algorithm are two eminent Quantum algorithms.

A. SHOR'S ALGORITHM

Shor's algorithm was developed in 1994 by American mathematician Peter Shor. It efficiently solves the fundamental problems in classical cryptography like integer factorization and discrete logarithm problems, which are considered hard for classical computers.

Shor's algorithm leverages quantum parallelism and the *quantum Fourier transform* (QFT) to efficiently determine the period (or order) of a modular exponentiation function, enabling the factoring of large composite numbers in polynomial time $O(logN)^3$—a problem that would require exponential time $O(\sqrt{N})$ by best known classical algorithm.

TABLE 5. Quantum gates with circuits and matrices [33], [34].

Gate	Circuit	Matrix	Description
Single Qubit Gates			
Identity		$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$	No changes in the state
Pauli X		$X = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	Quantum NOT gate: flips $\|0\rangle to \|1\rangle$ and vice versa
Pauli Y		$Y = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}$	Bit and phase flip: flips the state and adds a phase of i.
Pauli Z		$Z = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi} \end{bmatrix}$	Flips the phase of $\|1\rangle$ (adds a minus sign), leaves $\|0\rangle$ unchanged.
Hadamard		$H = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$	Superposition by transforming basis states: converts $\|0\rangle$ to $\frac{(\|0\rangle+\|1\rangle)}{\sqrt{2}}$ and $\|1\rangle$ to $\frac{(\|0\rangle-\|1\rangle)}{\sqrt{2}}$
Pauli X Rotation		$R_X(\theta) = \begin{bmatrix} cos(\theta/2) & -i\,sin(\theta/2)) \\ -i\,sin(\theta/2) & cos(\theta/2) \end{bmatrix}$	Performs a rotation around X axis by an angle θ
Pauli Y Rotation		$R_Y(\theta) = \begin{bmatrix} cos(\theta/2) & -sin(\theta/2)) \\ sin(\theta/2) & cos(\theta/2) \end{bmatrix}$	Performs a rotation around Y axis by an angle θ
Pauli Z Rotation		$R_Z(\theta) = \begin{bmatrix} exp(^{-i\theta}/_2) & 0 \\ -0 & exp(^{i\theta}/_2) \end{bmatrix}$	Performs a rotation around Z axis by an angle θ
Phase Gate-S		$S = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/2} \end{bmatrix}$	Adds a phase of $\frac{\pi}{2}$
Phase Gate-T		$T = \begin{bmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{bmatrix}$	Adds a phase of $\frac{\pi}{4}$
Two Qubit Gates			
CNOT Controlled X		$CX = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$	Entanglement with logic: Applies an X-gate to the target qubit only if the control qubit is $\|1\rangle$
Controlled Y		$CY = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & -i \\ 0 & 0 & i & 0 \end{bmatrix}$	Entanglement with logic: Applies an Y-gate to the target qubit only if the control qubit is $\|1\rangle$
Controlled Y		$CZ = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}$	Entanglement with logic: Applies an Z-gate to the target qubit only if the control qubit is $\|1\rangle$
SWAP		$SWAP = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$	Swaps two qubit states: Qubit 1 becomes Qubit 2, and vice versa. SWAP(q1,q2)=CNOT(q1,q2)→CNOT(q2,q1)→CNOT(q1,q2)
Three Qubit Gates			
Toffoli CCNOT, CCX, TOF		$CY = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$	Flips the target qubit only if the first two control qubits both are in the $\|1\rangle$

The algorithm operates in three fundamental stages: *Classical pre-processing, Quantum phase and Classical post-processing*.

The ***classical pre-processing*** (figure 18) in Shor's algorithm efficiently handles simple cases first. It checks if N is prime: the algorithm aborts since prime numbers have no non-trivial factors; if N is even (i.e., divisible by 2): it returns the factor 2; if N is a power of a prime: it return the base prime; if none apply, it assumes N is a product of distinct primes. It then picks a random a such that $1 < a < N$, and computes $gcd(a, N)$. If the GCD is greater than 1, it returns a non-trivial factor; otherwise, it proceeds to the quantum step.

The ***quantum part*** (figure 19) of Shor's algorithm finds the order r of a mod N, the smallest r such that $a^r \equiv 1 \bmod N$. This hard problem is solved efficiently using quantum phase estimation. The algorithm uses two quantum registers: one for phase estimation and one for modular exponentiation. After preparing a superposition and applying

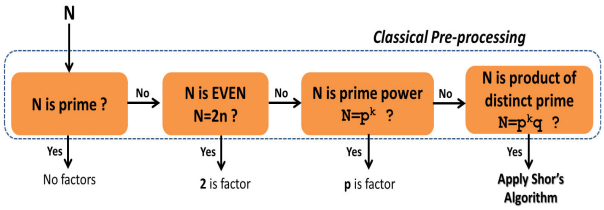


FIGURE 18. Shor's algorithm: Classical pre-processing stage.

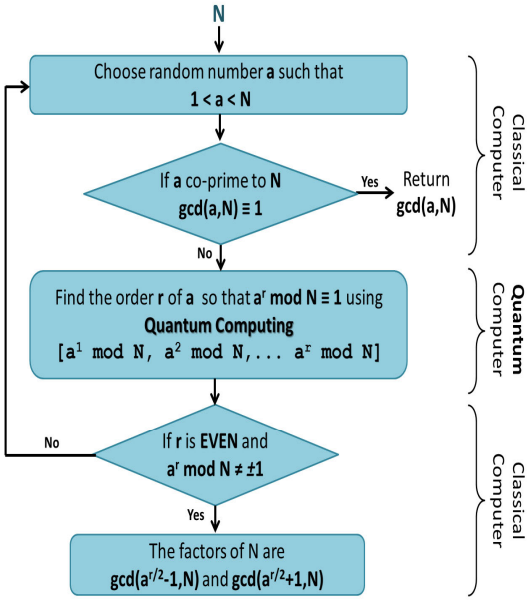


FIGURE 19. Shor's algorithm: Quantum stage.

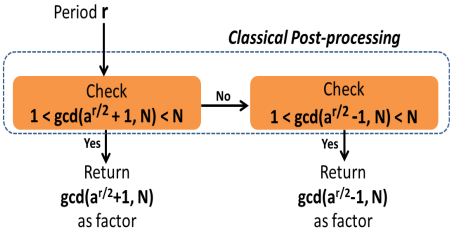


FIGURE 20. Shor's algorithm: Classical post-processing stage.

modular exponentiation, it uses an inverse Quantum Fourier Transform (QFT) and measures the result. The measured value is analyzed using continued fractions to approximate r. This step is exponentially faster than classical methods.

After finding the order r, the last part of algorithm ***classical post-processing*** (figure 20) uses classical steps to extract a factor of N. If r is even, it computes $a^{\frac{r}{2}}$ and checks $gcd(a^{(\frac{r}{2})} \pm 1, N)$. If either GCD yields a non-trivial factor, it is returned; otherwise, If r is odd, the attempt fails and may be retried with a new a. This step seizures the quantum result into an actual factor of N [35], [36].

B. GROVER'S ALGORITHM

Grover's algorithm is a quantum algorithm proposed by Lov Grover in 1996, that performs searches in unordered datasets exponentially quicker than compared to classical algorithms.

Algorithm: ***Shor's_Factorization(Integer N)***

// Input: integer number N for which want to find factors
// Output: factors of given number

1: Classical Pre-processing:
   i. Evaluate primarily of N: If yes, return no factors exist
   ii. Check if N is a prime power: If yes, return prime as result
   iii. Check if N is even: If yes, return 2 as a factor
   iv. N is multiplicative of prime or power of prime, go to step 4
2: Quantum Part(core part):
   i. Randomly choose an integer a such that $1 < a < N$
   ii. Compute $gcd(a, N)$: If $gcd(a, N) > 1$, return it as a factor.
   iii. Employ quantum computation to identify the period r of the function $f(x) = ax \bmod N$.
   - Prepare a quantum state representing superpositions of all x.
   - Apply the function $f(x) = ax \bmod N$ to create an entangled state.
   - Apply the QFT to extract the period r.
   - Measure the quantum state to get an estimate of r.
3: Classical Post-processing:
   i. If r is odd or $a^{\frac{r}{2}} \equiv \pm 1 \bmod N$, go back to step 5 and try a new a. Otherwise go to step 9
   ii. Compute $gcd(a^{\frac{r}{2}} \pm 1, N)$: These values are nontrivial factors of N.
   iii. Return the factors.

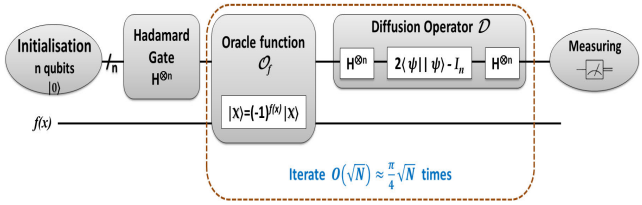


FIGURE 21. Grover's algorithm.

Compare to classical algorithms, the time complexity is reduced from $O(N)$ to $O(\sqrt{N})$ with Grover's algorithm, representing a quadratic speedup on a quantum computer [34].

Figure 21 illustrates overview of Grover's algorithm. The algorithm begins by initializing the n input qubits to the $|0\rangle$ state and transformed into a uniform superposition using Hadamard gates $H^{\otimes n}$ over all possible inputs. The oracle function Of marks the target state by inverting its phase. Then, the diffusion operator D (inversion about the mean) amplifies the amplitude of the marked state. This Grover iteration—comprising the oracle and diffusion steps—is repeated approximately $\frac{\pi}{4}\sqrt{N}$ times. A final measurement

on the input register yields the marked element with high probability [35], [37], [38].

Algorithm: ***Grover's_Algorithm(N Qubits)***
// Input: A quantum state space of size $N = 2n$
// Output: Search result

1: Initialization:
- Initialize all n-qubits to $|0\rangle : |0\rangle^{\otimes n}$
- Apply Hadamard gates to all qubits to create an equal superposition of all $N = 2n$ possible states:
$$|\psi\rangle = \frac{1}{\sqrt{N}} \sum_{0}^{(N-1)} |x\rangle$$

2: Grover iteration:
Iteratively apply the Oracle function O_f and then the Diffusion operator $\mathcal{D}$ approximately $\frac{\pi}{4}\sqrt{N}$ times to maximize the probability of measuring the correct result.
- Oracle function (quantum subroutine):
  · Flip the sign of the amplitude (Phase shift) of the target state $|x_0\rangle$ using quantum oracle Of function $O_f(|x\rangle) = (-1)\, f(x)\, |x\rangle$ Where, $f(x)$ is binary function, it changes the sign of amplitude of target state
$$f(x) = \begin{cases} 1, & \text{if } x = x_0 \text{ (target state)} \\ 0, & \text{otherwise} \end{cases}$$
  Only correct state (solution state) gets a negative sign.
$$f(x) = 1 \implies (-1)^1 = -1$$
$$f(x) = 0 \implies (-1)^0 = 1$$
- Grover Diffusion Operator (Amplitude Amplification):
  · Reflects the quantum state about the average amplitude.
$$D = 2\langle\psi||\psi\rangle - I$$
  This increases the probability amplitude of the correct state (solution state) and decreases the others.

3: Measurement:
Measure the quantum state. With high probability, the result will be the desired solution

Each Grover-iteration enhance the probability of identifying the correct solution and rotates the state vector closer to the solution. After $O(\sqrt{N})$ iterations, the correct answer dominates the superposition, making it highly likely to be observed upon measurement [39].

Heading beyond Shor's and Grover's algorithms, a few other notable quantum algorithms include the Deutsch-Jozsa, Bernstein-Vazirani, and Simon's algorithms, each showcasing quantum speedup for specific problem types. These algorithms tackle tasks like function classification, hidden string extraction, and XOR mask detection exponentially faster than classical methods. They exemplify how quantum principles can be leveraged for algorithmic efficiency beyond the well-known factoring and search problems.

C. DEUTSCH–JOZSA ALGORITHM

The Deutsch-Jozsa algorithm proposed by David Deutsch and Richard Jozsa in 1992, it stands as an early example of quantum algorithms showcasing a notable speedup over classical methods. Deutsch-Jozsa algorithm is a quantum algorithm designed to determine whether a given Boolean function $f : \{0,1\}^n \to \{0,1\}$ is constant (returns the same value for all inputs) or balanced. A balanced function outputs an equal number of $0s$ and $1s$, while a constant function outputs either all $0s$ or all $1s$. This is a decision problem where classically, in the worst case, $2^{n-1} + 1$ function evaluations are needed. The quantum version solves it with just a single query offering a significant speedup compared to classical algorithms. It can be derived from the quantum Fourier transform algorithm and has been used as an educational tool to illustrate fundamental qubit concepts and algorithmic challenges. The algorithm begins by initializing n input qubits in state $|0\rangle$ and an ancillary qubit in $|1\rangle$ then applies a Hadamard transform $H^{\otimes n+1}$, followed by a query to an oracle U_f that maps $|x\rangle|y\rangle \to |x\rangle\ |y \oplus f(x)\rangle$, and another Hadamard transformation. Upon measuring the first n qubits, a result of $|0\rangle^{\otimes n}$ indicates a constant function, while any other outcome implies it is balanced. This provides an exponential speedup over classical approaches [25], [34].

D. BERNSTEIN–VAZIRANI ALGORITHM

The Bernstein-Vazirani algorithm aims to determine a hidden binary string $s \in \{0,1\}^n$ encoded in a abstract function $f(x) = s \cdot x \bmod 2$, where $\cdot$ denotes the bitwise inner product. As the length of the hidden string increases, the classical algorithm becomes inefficient, even in a noiseless scenario. Classically, determining s requires n queries on $f(x)$, but Bernstein-Vazirani algorithm leverages quantum principles, finds s with fewer queries than classical algorithms, even in the presence of noise and imperfect equipment. The algorithm starts by placing the system in a uniform superposition via Hadamard gates: $H^{\otimes n}|0\rangle^{\otimes n} = \frac{1}{\sqrt{2^n}} \sum_X |x\rangle$. The oracle U_f is then applied as $U_f|x\rangle = (-1)^{f(x)}|x\rangle$, introducing a phase flip based on the secret string s. Applying another round of Hadamard transforms results in a final state $|s\rangle$, which is measured to reveal the hidden string in a single execution. This quantum solution is more robust against noise and becomes especially efficient as the string length grows [34].

E. SIMON'SS ALGORITHM

Simon's algorithm is a quantum algorithm used to find the hidden-period of a vectorial Boolean function with polynomial time complexity. Given a function $f : \{0,1\}^n \to \{0,1\}^n$, guaranteed to be 2-to-1 and periodic such that there exists a secret string s where $f(x) = f(y)$ if $x \oplus y = s$, the goal is to find s. Classically, this problem requires exponential time, but Simon's algorithm solves it in polynomial time. The algorithm prepares a superposition $\frac{1}{\sqrt{2^n}} \sum_x |x\rangle|0\rangle$, queries the function using an oracle $U_f|x\rangle|0\rangle = |x\rangle\ |f(x)\rangle$, then measures the second register, collapsing the system into a

state entangled over inputs that share the same output. A Hadamard transform is applied to the first register to yield a state orthogonal to s. Repeating the process yields enough linear equations to solve for s. The algorithm has applications in quantum cryptanalysis and has been used to study the autocorrelation and Walsh spectrum of Boolean functions. It has also found application in the design of lightweight encryption algorithms for IoT security and has been analysed for various use cases in cryptanalysis [34].

VI. POST QUANTUM CRYPTOGRAPHY

Traditional cryptography faces an existential challenge from the rapid progress in quantum computing. This underscores the critical need of shift to quantum-resistant solutions like post-quantum cryptography (PQC) to ensure data reliability across various sectors. Security of traditional encryption hinges on the challenge of integer factorization, used in RSA, and the discrete logarithm problem, used in ECC. Shor's algorithm has a substantial effect on asymmetric-key cryptography as it efficiently solves integer factorization, rendering RSA and ECC susceptible. Gitonga, simulations show that Shor's algorithm could break RSA-2048 and ECC-256 encryption using 4000 qubits and 2500 qubits with millions of operation respectively [22].

Post-quantum cryptography focuses on developing cryptographic algorithms that remain secure even against quantum computer attacks. These algorithms are resistant to threats like Shor's algorithm, rely on diverse mathematical foundations, e.g., lattice-based, hash-based, and code-based problems, and aim to future-proof data security in expectancy of quantum computing breakthroughs [40], [41].

A. CODE-BASED CRYPTOGRAPHY

Code-based cryptographic algorithms are among the earliest and most robust approaches to post-quantum cryptography. Decoding random linear error-correcting codes is a challenge so immense that even quantum computers can't crack it. These algorithms form a crucial part of quantum-resistant encryption, particularly in public key encryption (PKE) and key encapsulation mechanisms (KEMs). Several code-based schemes have gained attention, including BIKE (Bit Flipping Key Encapsulation), HQC (Hamming Quasi-Cyclic), and Classic McEliece [42], [43], [44].

1) McEliece

McEliece is one of the most prominent code-based public Key encryption algorithms, named after Robert J. McEliece, who proposed the system in 1978. Its foundation lies in the computational hardness of decoding a general linear error-correcting code, for which no classical or quantum vulnerability has been found. McEliece remains a very strong candidate for secure post-quantum encryption, especially for applications where high security is critical and storage space is less constrained. Its proven long-term security and resistance to quantum attacks make it a valuable part of the post-quantum cryptographic toolbox. Classic McEliece is updated and optimized variant of McEliece [42], [45].

The major drawback of McEliece is its very large public key size, which can range from hundreds of kilobytes to megabytes, making it less practical for certain applications [42], [46].

Algorithm: ***McEliece_Key Generation(n,k,t)***
//Input: Parameters for a Goppa code (length n, dimension k, error-correcting capacity t)
//Output: Public key G', Private key (G,S,P)

1: Select a binary Goppa code with:
   Generator matrix $G \in F_2^{(k\times n)}$ where, $F_2^{(k\times n)}$ finite binary field with 0, 1 k number of rows and n number of columns
   Error-correcting capability for up to t errors
2: Generate:
   S: random invertible $k \times k$ scrambling matrix
   P: random $n \times n$ permutation matrix
3: Compute public generator matrix:
   $G' = S \cdot G \cdot P$
4: **return** public-key: G' private-key: (G, S, P)

Algorithm: ***McEliece_Encryption(m,G')***
//Input: Message $m \in F_2^k$, Public key G'
//Output: Ciphertext c

1: Generate a random error vector $e \in F_2^n$ with weight $\leq t$
2: Compute ciphertext:
   $c = m \cdot G' + e$
3: end ciphertext c to the receiver

Algorithm: ***McEliece_Decryption(c,(S,G,P))***
//Input: Ciphertext c, Private key (S,G,P)
//Output: Original message m

1: Compute:
   $c' = c \cdot P^{-1}$
2: Use Goppa decoding algorithm on c' to retrieve:
   $m' = Decode(c', G)$
3: Recover original message:
   $m = m' \cdot S^{-1}$

McEliece uses Goppa codes: mathematically robust type of error-correcting code which is basis for its strong security guarantees;

B. LATTICE-BASED CRYPTOGRAPHY

Lattice-based cryptography stands out as a robust and efficient solution for securing digital communications against powerful quantum computers in the post-quantum era. They rely on the hardness of mathematical problems in high-dimensional lattices, such as the Short Integer Solution (SIS) and Shortest Vector Problem (SVP), which are believed to be resistant to quantum attacks. These problems are difficult to

solve even with quantum algorithms like Shor's, making them ideal for securing sensitive data in the post-quantum era [44], [47], [48], [49].

A notable example is the lattice-based encryption scheme built using ElGamal's principles. It offers strong security, simple construction, and efficient performance, especially for small messages. Lattice-based systems also support encryption, signatures, and key exchange protocols. NIST has selected lattice-based algorithms like CRYSTALS-Kyber: for encryption and CRYSTALS-Dilithium and FALCON: for digital signatures for standardization due to their quantum resistance [40], [43], [47].

1) CRYSTALS-KYBER: CRYPTOGRAPHIC SUITE FOR ALGEBRAIC LATTICES–KYBER

NIST standardized CRYSTALS-Kyber as a lattice-based post-quantum Key Encapsulation Mechanism (KEM) in 2022. It is built on the Module Learning With Errors (M-LWE) problem and offers three security levels—Kyber-512, Kyber-768, and Kyber-1024—corresponding roughly to AES-128, AES-192, and AES-256 strength. Kyber is designed to resist both classical and quantum attacks and uses a variant of the Fujisaki–Okamoto transform to achieve Indistinguishability under Chosen Ciphertext Attack- IND-CCA2 security. Due to its efficiency and quantum resistance, Kyber is recommended for use in hybrid encryption systems and has been adopted by the NSA for national security applications [40], [42], [45], [50], [51]. The CRYSTALS-Kyber design offers efficient key generation, encryption, and decryption, with low power and memory consumption, making it suitable for various hardware platforms like RISC-V and FPGAs [35].

Algorithm: ***CRYSTALS-Kyber_Key Generation (Random seed)***
//Input: Random seed
//Output: Public-key (PKey), Secret-key (SKey)

1: Generate a random seed.
2: Use the seed to create a public matrix A (in module-lattice form).
3: Sample secret vector s and error vector e from a noise distribution.
4: Compute:
$$t = A \cdot s + e$$
5: **return** Public key: PKey $= (A, t)$ Secret key: SKey $= s$

Algorithm: ***CRYSTALS-Kyber_Encapsulation (Public-key PKey, message M)***
//Input: Public-key (PKey)
//Output: Ciphertext (c), Shared-key (S)

1: Sample ephemeral secret r, and errors e_1, e_2, e_3.
2: Compute:
$$u = A \cdot r + e_1$$
$$v = t \cdot r + e2 + M \cdot \lfloor q/2 \rfloor$$
where m is the message (shared key seed), encoded in lattice form.
3: **return** Ciphertext: c = (u, v)
Shared key: derived from m

Algorithm: ***CRYSTALS-Kyber_Decapsulation (Dec)***
//Input: Ciphertext (ct), Secret key (SKey = s)
//Output: Shared key (K)

1: Use secret key s to compute:
$$M' = v - u \cdot s$$
2: Recover the message M from M'.
3: Use M to derive the shared key.

2) CRYSTALS-DILITHIUM: CRYPTOGRAPHIC SUITE FOR ALGEBRAIC LATTICES-DILITHIUM

CRYSTALS-Dilithium is a post-quantum digital signature scheme that was standardized by NIST in July 2022. It is built on the Module Learning with Errors (MLWE) and Module Short Integer Solution (MSIS) problems. Using the Fiat–Shamir transformation, it ensures strong protection in the Quantum Random Oracle Model (QROM). Dilithium generates short public keys and signatures and is efficient in signing and verification processes. It balances performance across key size, speed, and resistance to quantum threats. Implementations of Dilithium on hardware platforms like FPGA and RISC-V have shown promising efficiency, making it a leading candidate standardized by NIST for post-quantum digital signatures [40], [42], [45], [52].

Algorithm: ***CRYSTAL-Dilithium_KeyGeneration(Random seed)***
//Input: Random seed
//Output: Public key (P_{key}), Secret key (S_{key})

1: Generate public matrix A using a seed.
2: Sample small secret vectors s1, s2 from centered binomial distribution.
3: Compute: $t = A \cdot s_1 + s_2$
4: **return** Public key: $P_{key} = (A, t)$
Secret key: $S_{key} = (s_1, s_2)$

3) FALCON-FAST-FOURIER LATTICE-BASED COMPACT SIGNATURES OVER NTRU

FALCON is a lattice and GPV framework (Gentry–Peikert–Vaikuntanathan)-selected by NIST for a quantum-resistant digital signature standard. FALCON is noted for its performance advantages in certain scenarios, particularly where compact and efficient digital signatures are essential. It is designed to offer strong security guarantees grounded in the hardness of lattice problems, specifically leveraging the NTRU lattice structure, making it resilient to attacks by both classical and quantum computers. FALCON is recognized for its potential in performance-critical applications [40], [42], [45], [47].

Algorithm: ***CRYSTAL-Dilithium_Signing(Message M, Secret-key sk)***
//Input: Message M, Secret-key S_{key}
//Output: Signature $\sigma = (z, c)$

1: Sample random vector y.
2: Compute Intermediate vector
$w = A \cdot y$
3: Hash message and w to get challenge:
$c = H(w, M)$
4: Compute main signature part
$z = y + c \cdot s_1$
5: Output signature$\sigma = (z, c)$

Algorithm: ***CRYSTAL-Dilithium_Verification()***
//Input: Message M, Public key $= (A, t)$, Signature $\sigma = (z, c)$
//Output: Accept/Reject

1: Compute
$w' = A \cdot z - c \cdot t$
2: Compute hash:
$c' = H(w', M)$
3: Accept if $c = c'$ and z is within bounds, else Reject

Algorithm: ***FALCON_KeyGeneration()***
//Input: - -
//Output: public and private key pair.

1: Choose two small polynomials f and g from a discrete Gaussian distribution.
2: Construct the lattice basis:
Use NTRU equation: $f * G - g * F = q \bmod \phi$
3: Compute a public key:
$h = (g/f) \bmod q$ in the NTRU ring.
4: **return**
Secret key: (f, g, F, G) Must satisfy the NTRU relation for trapdoor lattice
Public key: h

Algorithm: ***FALCON_Signing(message m)***
//Input: Message m
//Output: signature σ for a message m.

1: Hash the message:
Compute message digest $c = Hash(m)$, mapped to a point in the NTRU lattice space.
2: Trapdoor Sampling:
Use fast Fourier Gaussian sampling (via FFT) to find a lattice vector z such that:
z is short
$z \approx$ preimage of c under the public matrix B
3: Signature $\sigma = z$

Algorithm: ***FALCON_Verification(message m)***
//Input: message m
//Output: Verify the signature σ is valid for message m.

1: Hash the message again:
Compute $c = Hash(m)$
2: Check Signature:
Verify that σ is a short vector (i.e., within norm bound)
Check:
$$B \cdot \sigma \equiv c \bmod q$$
Confirm that signature has acceptable norm (tight security bound)

C. HASH-BASED CRYPTOGRAPHY

Hash based cryptographic algorithms are considered to be among the most quantum-resilient, unlike systems vulnerable to Shor's algorithm on a quantum computer, their security doesn't hinge on numerical problems such as factoring or discrete logarithms. They relies solely on the security of hash functions—such as SHA-256—for constructing secure cryptographic primitives like digital signatures. Since hash functions are relatively simple and well-analysed compared to more complex algebraic structures, their security assumptions are viewed as conservative and trustworthy [43], [47].

1) SPHINCS+-STATELESS PRACTICAL HASH-BASED INTERNET CRYPTOGRAPHY SCHEME

SPHINCS+ is a hash-based, stateless digital signature algorithm it provides strong security in the post quantum era. SPHINCS+ relies entirely on cryptographic hash functions for its security. As hash functions are among the most conservative cryptographic primitives, SPHINCS+ remains relatively resistant compared to number-theory-based systems vulnerable to Shor's algorithm.

The stateless design of SPHINCS+ eliminates the need for maintaining a record of previously used secret keys, making it simpler to implement securely in constrained or high-assurance environments. However, the trade-offs in performance and key/signature sizes make it less suitable for applications where speed or bandwidth is critical [42], [45], [47].

Forest of Random Subsets-FORS and Winternitz One-Time Signature-WOTS+ are key components of the SPHINCS+ signature scheme. FORS (Forest of Random Subsets) is a few-time signature scheme used in SPHINCS+ to sign the hashed message using multiple small Merkle trees. It provides fast and compact signatures by selecting specific leaves based on the message hash and authenticating them. WOTS+ is a simple, stateless, and post-quantum secure signature scheme that relies solely on hash functions, avoiding complex mathematical assumptions. WOTS+ parameters: security parameter n(e.g., 256 bits), Winternitz parameter w(e.g., 16), and derived length parameters l = l1 + l2: total number of chains i.e., total signature size, depends on w. l1: number of message-dependent chains, l2: number of checksum chains.

Comparison of classical and quantum cryptographic algorithms is shown in Table 6.

TABLE 6. Comparison between classical and Quantum cryptographic algorithms.

Parameter	Classical cryptography	Quantum cryptography
Security Basis	Based on ***computational hardness of mathematics*** - factoring, discrete log, elliptic curves	Based on ***laws of quantum mechanics*** – like Heisenberg's uncertainty principle, no-cloning theorem
Security Guarantee	***Computational security*** - can be broken with efficient algorithms and faster computers with enough resources.	***Unconditional security*** - guaranteed by the fundamental principles of quantum mechanics, making it independent of any computational resources or power.
Key Distribution	Key sharing is performed through algorithms like ***RSA, ECC, or Diffie–Hellman***. The safety relies on the difficulty of solving large mathematical problems.	Quantum cryptography ***Quantum Key Distribution (QKD)*** using quantum particles for secure key sharing. Eavesdropping disturbs the quantum states, which can be immediately detected by the users.
Error Handling	Errors are handled using well-established ***redundancy*** and ***error-correction codes*** (e.g., CRC, Hamming, LDPC), with low complexity and high efficiency.	Errors require ***Quantum Error Correction (QEC)*** or privacy amplification since quantum states are fragile and cannot be copied, making error handling more complex.
Efficiency / Speed	***Very efficient*** — RSA, ECC, AES run fast on standard hardware, suitable for high-throughput communication.	***Less efficient*** — QKD and quantum protocols are slower due to photon transmission, noise, and error correction overhead.
Resistance to Quantum Attacks	***Vulnerable*** - Shor's algorithm breaks RSA/ECC; Grover reduces symmetric cipher strength.	***Resistant*** — security guaranteed by quantum mechanics, independent of computational hardness.
Cost	***Low-cost*** - since it is software-based, easily deployable over current internet and devices.	***High-cost*** - as it requires specialized quantum hardware and infrastructure.

Algorithm: ***SPHINCS+_KeyGeneration()***
//Input: Security parameter, Hash function
//Output: Public key(P_{Key}), Secret key(S_{Key})
// FORS - Forest of Random Subsets
// WOTS - Winternitz One-Time Signature

1: Generate:
S_{Key_seed}: secret seed - used for secret key for WOTS+ and randomness FORS
P_{Key_seed}: public seed - used to derive public key elements
S_{Key_prf}: Pseudo-Random Function – used for randomized signing
2: Construct the top-level Merkle tree from WOTS+, whose root becomes part of the public key
3: Hash function is used to:
Compress WOTS+ outputs
Hash internal nodes of the Merkle tree upto the root
4: **return**
$S_{Key} = (S_{Key_seed}, P_{Key_seed}, S_{Key_prf}, root)$
$P_{Key} = (P_{key_seed}, root)$

Algorithm: ***SPHINCS+_Signing(Message*** M***, Secret-key*** S_{Key}***)***
//Input: Message M, Secret-key (S_{Key})
//Output: Signature σ
//PRF - Pseudo-Random Function

1: Randomization: Derive randomness from SK and message M (to prevent signature re-use).
$R = PRF_{SKey_prf}(M)$
2: Message Hashing: Derive tree index and leaf index for FORS signature
$H = Hash(R, PK, M)$
3: FORS Signing:
Use FORS to sign a hashed message based on a few subkeys.
4: Hypertree Traversal:
WOTS Signing Use WOTS+ to sign FORS outputs (FORS signature, FORS public key).
Construct authentication paths up a hypertree of Merkle trees.
5: Signature: Combine all: randomness + FORS signature + WOTS+ signature + auth paths.
σ (R, FORS sig, WOTS+ sigs, Merkle auth paths)

VII. QUANTUM KEY DISTRIBUTION-QKD

For an OTP to be truly unbreakable, its key should be truly random, at least as long as the message, and never reused—either fully or partially. Quantum Key Distribution (QKD) offers a ground-breaking approach to generating such keys, ensuring communication is fundamentally secure. Quantum key distribution uses two channels(Figure 22): a quantum channel for transmitting quantum information and a classical channel for public communication. In the quantum channel, individual photons are sent; each encoded using polarization to represent bits of the key.

The figure 23 shows the QKD process, where qubits are distributed through a quantum channel, followed by

Algorithm: ***SPHINCS+_Verification(Message M, Signature σ, Public-key P_{Key})***
//Input: Message M, Signature σ, Public key (P_{Key})
//Output: Accept / Reject

1: Recompute the message hash H from the R,M.
2: Reconstruct the FORS public key from its signature.
3: Verify WOTS+ signatures and use Merkle authentication paths to reconstruct the root.
4: Compare computed Merkle root with the public key's root.
5: If equal → Accept, else → Reject

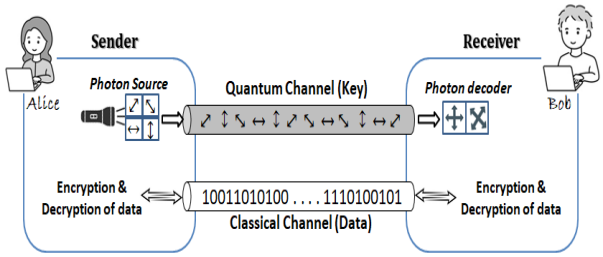


FIGURE 22. QKD network architecture.

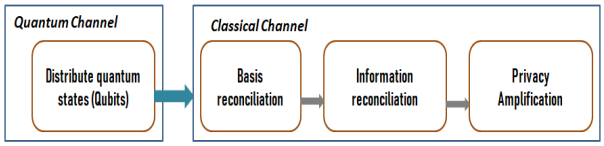


FIGURE 23. Common underlying structure of QKD [53].

basis reconciliation, information reconciliation, and privacy amplification over a classical channel.

Just as light exhibits particle-wave duality, these two security approaches employ a dual perspective, by conceptualizing light as either discrete particles or continuous waves to ensure security. Discrete Variable (DV) and Continuous Variable (CV) are encoding types used in QKD protocols. DV encodes information using photon polarization or phase, CV uses continuous quadrature measurements, each offering different advantages in implementation and security [53].

A. DV-QKD

DV-QKD encodes quantum information using discrete properties of photons, such as polarization or phase, relying on single-photon detection. Which uses the continuous wave nature of light, and uses individual photon states, and security arises from the no-cloning theorem and the uncertainty principle. DV-QKD is well-developed and experimentally validated, but its challenges include requiring single-photon sources and highly sensitive photon detectors. It typically uses protocols like BB84 or E91, where quantum bits (qubits) are transmitted as individual photons in specific polarization states [54].

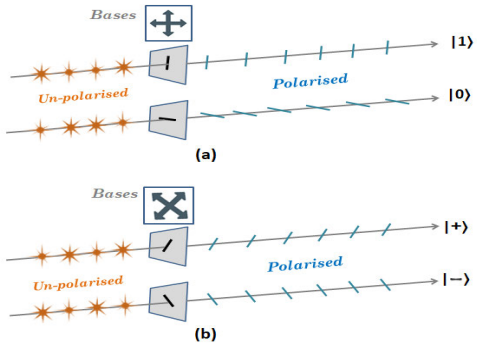


FIGURE 24. Measurement bases and photon polarization states: (a) Rectilinear $|0\rangle$, $|1\rangle$ and (b) Diagonal $|+\rangle \implies$ binary 0, $|-\rangle \implies$ binary 1.

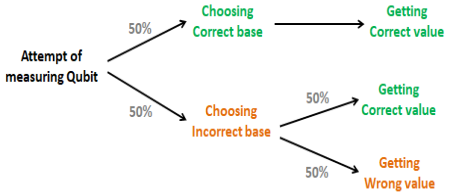


FIGURE 25. Probabilities of measuring Qubits.

1) BB84

Building on their work with "quantum multiplexing" channel theory, C.H. Bennett and G. Brassard published the ground breaking BB84 QKD protocol in 1984, address the challenge of key distribution in classical cryptography.

In this protocol, sender say Alice, prepares quantum bits using two non-orthogonal bases: rectilinear ($|0\rangle$, $|1\rangle$) or diagonal($|+\rangle$, $|-\rangle$), typically denoted as +/x see figure 24. Each basis encodes a classical bit ($0 or 1$). On the other side, receiver say Bob, randomly selects the basis for measuring these qubits [55].

Due to quantum mechanics, if Bob measures in the correct basis, he obtains the same bit Alice sent; if he chooses the wrong basis, the result is random. Figure 25: When measuring a qubit, there's a 50% chance of choosing the correct or incorrect basis. Correct bases yielding the correct value, where incorrect bases, there's a further 50% chance of getting either the correct or incorrect value.

After the broadcast, Alice and Bob openly compare which basis they used (but not the measurement outcomes) and discard all mismatched cases. To ensure security, they check a subset of bits for errors (due to noise or eavesdropping). If the Quantum Bit Error Rate (QBER) is below a threshold, they perform error correction and privacy amplification, yielding a final shared secret key. The BB84 protocol is secure because any eavesdropper (Eve) will disturb the quantum states and introduce detectable errors due to the no-cloning theorem and the Heisenberg uncertainty principle(figure 26).

B. CV-QKD

Continuous-Variable QKD (CV-QKD) utilizes the wave properties of light, like amplitude and phase that leverages

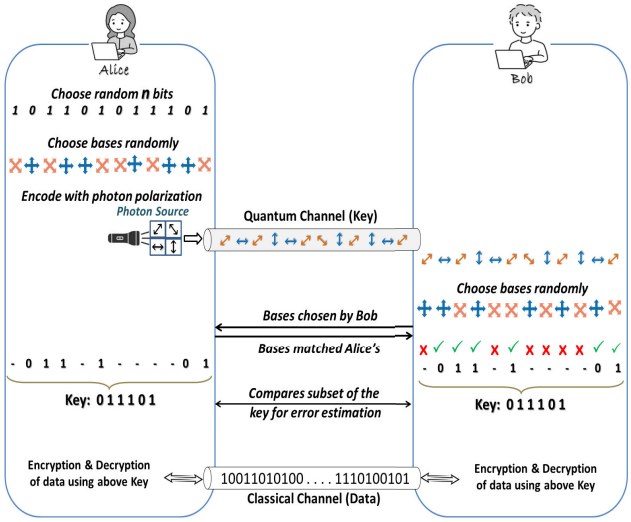


FIGURE 26. BB84 protocol basic scheme.

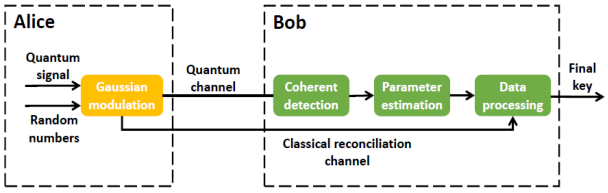


FIGURE 27. The schematic diagram of the GG02 protocol.

the principles of quantum mechanics to ensure information-theoretic security. CV-QKD encodes information corresponding to the quadrature of the electromagnetic field. A basic advantage of CV-QKD is its compatibility with current telecommunication infrastructure, as it can utilize commercial lasers and homodyne detectors. This compatibility has led to significant advancements, moving CV-QKD from proof-of-principle laboratory demonstrations to in-field implementations and technological prototypes. Advances in protocols, security analysis, and hardware integration have enabled high-speed, long-distance, and chip-based implementations suitable for modern quantum-secure communication networks [53], [56], [57].

1) GG02

The GG02 protocol, a notable continuous-variable quantum key distribution (CV-QKD) protocol, was developed by Frederic Grosshans and Philippe Grangier and introduced in 2002. Its "GG" designation directly references its inventors. GG02 uses the continuous quadrature variables—specifically, the amplitude and phase—of light to encode information, relying on coherent states. It is known for its practicality, as it uses standard telecom components like lasers and homodyne detectors, and allows high key rates over metropolitan-scale distances [58].

The GG02 quantum key distribution (QKD) protocol operates using coherent states of light and Gaussian modulation to securely establish a private key between two parties, Alice and Bob (figure 27). First, Alice generates two random numbers from a Gaussian distribution to represent the amplitude (X) and phase (P) quadratures of a coherent state. She then prepares a coherent state with these values and sends it through an optical channel to Bob. Upon receiving the state, Bob randomly chooses to measure either the amplitude or phase quadrature using homodyne detection and records the result. He then informs Alice, via a public channel, of the quadrature measurement for each pulse. Alice discards the unmeasured quadrature data and keeps only the relevant part. At this stage, Alice and Bob have correlated sets of continuous variables. They apply a sliced reconciliation protocol to convert these into matching discrete bit strings, followed by privacy amplification to eliminate any partial information that an eavesdropper (Eve) might have obtained. The final result is a shared, secure secret key, with the security guaranteed by the quantum no-cloning theorem, provided the channel loss is less than 3 dB with transmission efficiency above 50% [58].

Quantum key distribution (QKD) faces several practical challenges, including the need for expensive and specialized hardware, limited transmission range, and sensitivity to interference that can cause communication failures. It also relies on classical cryptographic methods for authentication. Due to these constraints, QKD is currently viable only for specific use cases, while symmetric key systems and post-quantum cryptography remain more practical and scalable solutions for achieving quantum-safe security [59].

VIII. CONCLUSION

The evolution of cryptographic algorithms from the classical to quantum paradigms marks a significant milestone in securing digital communication. Classical cryptography, grounded in mathematical complexity, has served as the strength of secure systems for many years. However, the emergence of quantum computing challenges the assumptions underlying many of these schemes, particularly those based on factoring and discrete logarithms. Quantum cryptography, particularly the quantum key distribution (QKD), offers fundamentally new security guarantees based on the principles of quantum mechanics. This review has highlighted the strengths and limitations of both classical and quantum approaches, emphasizing the need for post-quantum cryptography and the continued development of quantum-resistant algorithms. As quantum technologies mature, a hybrid model that combines the practicality of classical systems with the security advantages of quantum methods may emerge as the most viable path forward. Continued research and standardization efforts are essential to ensure robust and future-proof security in the quantum era.

REFERENCES

[1] L. Zhang, Y. Fang, J. Xu, and Y. Shen, "Computer network security strategy based on data encryption technology," in *Proc. IEEE 15th Int. Conf. Comput. Intell. Commun. Netw. (CICN)*. Thailand: IEEE, Dec. 2023, pp. 387–393, doi: 10.1109/cicn59264.2023.10402235.

[2] A. W. Dent, "Choosing key sizes for cryptography," *Inf. Secur. Tech. Rep.*, vol. 15, no. 1, pp. 21–27, Feb. 2010, doi: 10.1016/j.istr.2010.10.006.
[3] M. Faheem, S. Jamel, A. Hassan, Z. A., N. Shafinaz, and M. Mat, "A survey on the cryptographic encryption algorithms," *Int. J. Adv. Comput. Sci. Appl.*, vol. 8, no. 11, pp. 333–344, 2017, doi: 10.14569/ijacsa.2017.081141.
[4] M. K. Mageshwaran and P. Sri Ramya, "Data protection in the era of big data advanced encryption standard VS data encryption standard," in *Proc. 2nd Int. Conf. Comput. Characterization Techn. Eng. Sci. (IC3TES)*. Lucknow, India: IEEE, Nov. 2024, pp. 1–5, doi: 10.1109/ic3tes62412.2024.10877503.
[5] S. Srilaya and S. Velampalli, "Performance evaluation for DES and AES algorithms- an comprehensive overview," in *Proc. 3rd IEEE Int. Conf. Recent Trends Electron., Inf. Commun. Technol. (RTE-ICT)*, Bangalore, India, May 2018, pp. 1264–1270, doi: 10.1109/RTE-ICT42901.2018.9012536.
[6] D. S. A. Elminaam, H. M. A. Kader, and M. M. Hadhoud, "Evaluating the performance of symmetric encryption algorithms," *Int. J. Netw. Secur.*, vol. 10, no. 3, pp. 216–222, May 2010.
[7] H. Muhammad Waseem, M. Asfand Hafeez, S. Ahmad, B. David Deebak, N. Munir, A. Majeed, and S. Oun Hwang, "Constructing highly nonlinear cryptographic balanced Boolean functions on learning capabilities of recurrent neural networks," *IEEE Access*, vol. 12, pp. 150255–150267, 2024, doi: 10.1109/ACCESS.2024.3477260.
[8] D. F. García, "Performance evaluation of advanced encryption standard algorithm," in *Proc. 2nd Int. Conf. Math. Comput. Sci. Ind. (MCSI)*, Sliema, Malta, Aug. 2015, pp. 247–252, doi: 10.1109/MCSI.2015.61.
[9] E. Barker, "Recommendation for key management part 1: General," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, Tech. Rep. NIST SP 800-57pt1r4, Jan. 2016, doi: 10.6028/NIST.SP.800-57pt1r4.
[10] R. Imam, Q. M. Areeb, A. Alturki, and F. Anwer, "Systematic and critical review of RSA based public key cryptographic schemes: Past and present status," *IEEE Access*, vol. 9, pp. 155949–155976, 2021, doi: 10.1109/ACCESS.2021.3129224.
[11] P. Chaudhury, S. Dhang, M. Roy, S. Deb, J. Saha, A. Mallik, S. Bal, S. Roy, M. K. Sarkar, S. Kumar, and R. Das, "ACAFP: Asymmetric key based cryptographic algorithm using four prime numbers to secure message communication. A review on RSA algorithm," in *Proc. 8th Annu. Ind. Autom. Electromechanical Eng. Conf. (IEMECON)*. Bangkok, Thailand: IEEE, Aug. 2017, pp. 332–337, doi: 10.1109/IEMECON.2017.8079618.
[12] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, Feb. 1978, doi: 10.1145/359340.359342.
[13] Y. Yan, "The overview of elliptic curve cryptography (ECC)," *J. Phys. Conf. Ser.*, vol. 2386, no. 1, Dec. 2022, Art. no. 012019, doi: 10.1088/1742-6596/2386/1/012019.
[14] S. Ullah, J. Zheng, N. Din, M. T. Hussain, F. Ullah, and M. Yousaf, "Elliptic curve cryptography; applications, challenges, recent advances, and future trends: A comprehensive survey," *Comput. Sci. Rev.*, vol. 47, Feb. 2023, Art. no. 100530, doi: 10.1016/j.cosrev.2022.100530.
[15] H. Ren, Y. Wang, Q. Xie, and H. Yang, "A novel method for one-way hash function construction based on spatiotemporal chaos," *Chaos, Solitons Fractals*, vol. 42, no. 4, pp. 2014–2022, Nov. 2009, doi: 10.1016/j.chaos.2009.03.168.
[16] S. A. Waheeb, N. A. Khan, and X. Shang, "An efficient sentiment analysis based deep learning classification model to evaluate treatment quality," *Malaysian J. Comput. Sci.*, vol. 35, no. 1, pp. 1–15, Jan. 2022, doi: 10.22452/mjcs.vol35no1.1.
[17] *Secure Hash Standard*, National Institute of Standards and Technology (U.S.), Washington, DC, USA, 2015, doi: 10.6028/nist.fips.180-4.
[18] L. E. Almeida, B. A. Fernández, D. Zambrano, A. I. Almachi, H. B. Pillajo, and S. G. Yoo, "One-time passwords: A literary review of different protocols and their applications," in *Advanced Research in Technologies, Information, Innovation and Sustainability* (Communications in Computer and Information Science), vol. 1936, T. Guarda, F. Portela, and J. M. Diaz-Nafria, Eds., Cham, Switzerland: Springer Nature, 2024, pp. 205–219, doi: 10.1007/978-3-031-48855-9_16.
[19] E. Erdem and M. T. Sandikkaya, "OTPaaS—One time password as a service," *IEEE Trans. Inf. Forensics Security*, vol. 14, no. 3, pp. 743–756, Mar. 2019, doi: 10.1109/TIFS.2018.2866025.
[20] S. Babkin and A. Epishkina, "Authentication protocols based on one-time passwords," in *Proc. IEEE Conf. Russian Young Researchers Electr. Electron. Eng. (EIConRus)*. Saint Petersburg and Moscow, Russia: IEEE, Jan. 2019, pp. 1794–1798, doi: 10.1109/EICONRUS.2019.8656839.
[21] D. Ramakrishna and M. Ali Shaik, "A comprehensive analysis of cryptographic algorithms: Evaluating security, efficiency, and future challenges," *IEEE Access*, vol. 13, pp. 11576–11593, 2025, doi: 10.1109/ACCESS.2024.3518533.
[22] C. K. Gitonga, "The impact of quantum computing on cryptographic systems: Urgency of quantum-resistant algorithms and practical applications in cryptography," *Eur. J. Inf. Technol. Comput. Sci.*, vol. 5, no. 1, pp. 1–10, Jan. 2025, doi: 10.24018/compute.2025.5.1.146.
[23] P. C. Nwaga and S. Nwagwughiagwu, "Exploring the significance of quantum cryptography in future network security protocols," *World J. Adv. Res. Rev.*, vol. 24, no. 3, pp. 817–833, Dec. 2024, doi: 10.30574/wjarr.2024.24.3.3733.
[24] H. A. Bhat, F. A. Khanday, B. K. Kaushik, F. Bashir, and K. A. Shah, "Quantum computing: Fundamentals, implementations and applications," *IEEE Open J. Nanotechnol.*, vol. 3, pp. 61–77, 2022, doi: 10.1109/OJNANO.2022.3178545.
[25] Z. Yang, M. Zolanvari, and R. Jain, "A survey of important issues in quantum computing and communications," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 2, pp. 1059–1094, 2nd Quart., 2023, doi: 10.1109/COMST.2023.3254481.
[26] M. Coccia, S. Roshani, and M. Mosleh, "Evolution of quantum computing: Theoretical and innovation management implications for emerging quantum industry," *IEEE Trans. Eng. Manag.*, vol. 71, pp. 2270–2280, 2024, doi: 10.1109/TEM.2022.3175633.
[27] P. Busch, T. Heinonen, and P. Lahti, "Heisenberg's uncertainty principle," *Phys. Rep.*, vol. 452, no. 6, pp. 155–176, Nov. 2007, doi: 10.1016/j.physrep.2007.05.006.
[28] I. Giroti and M. Malhotra, "Quantum cryptography: A pathway to secure communication," in *Proc. 6th Int. Conf. Comput. Syst. Inf. Technol. Sustain. Solutions (CSITSS)*. Bangalore, India: IEEE, Dec. 2022, pp. 1–6, doi: 10.1109/CSITSS57437.2022.10026388.
[29] F. L. Marquezino and R. R. M. Junior, "Considerations on classical and quantum bits," Centro Brasileiro de Pesquisas FÍsicas (CBPF), Rio de Janeiro, Brazil, Tech. Rep. CBPF-MO-003/04, 2004.
[30] S. R. Hasan, M. Z. Chowdhury, Md. Saiam, and Y. M. Jang, "Quantum communication systems: Vision, protocols, applications, and challenges," *IEEE Access*, vol. 11, pp. 15855–15877, 2023, doi: 10.1109/ACCESS.2023.3244395.
[31] A. Oukaira, "Quantum hardware devices (QHDs): Opportunities and challenges," *IEEE Access*, vol. 13, pp. 98229–98241, 2025, doi: 10.1109/ACCESS.2025.3576216.
[32] A. S. Cacciapuoti, M. Caleffi, R. Van Meter, and L. Hanzo, "When entanglement meets classical communications: Quantum teleportation for the quantum internet," *IEEE Trans. Commun.*, vol. 68, no. 6, pp. 3808–3833, Jun. 2020, doi: 10.1109/TCOMM.2020.2978071.
[33] J. Ur Rehman, M. S. Ulum, A. W. Shaffar, A. A. Hakim, Mujirin, Z. Abdullah, H. Al-Hraishawi, S. Chatzinotas, and H. Shin, "Evolutionary algorithms and quantum computing: Recent advances, opportunities, and challenges," *IEEE Access*, vol. 13, pp. 16649–16670, 2025, doi: 10.1109/ACCESS.2025.3530952.
[34] M. A. Shafique, A. Munir, and I. Latif, "Quantum computing: Circuits, algorithms, and applications," *IEEE Access*, vol. 12, pp. 22296–22314, 2024, doi: 10.1109/ACCESS.2024.3362955.
[35] Z. Yang, H. Alfauri, B. Farkiani, R. Jain, R. D. Pietro, and A. Erbad, "A survey and comparison of post-quantum and quantum blockchains," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 967–1002, 2nd Quart., 2024, doi: 10.1109/COMST.2023.3325761.
[36] P. W. Shor, "Algorithms for quantum computation: Discrete logarithms and factoring," in *Proc. 35th Annu. Symp. Found. Comput. Sci.* Santa Fe, NM, USA: IEEE Comput. Soc. Press, pp. 124–134, doi: 10.1109/SFCS.1994.365700.
[37] F. Ablayev, M. Ablayev, J. Z. Huang, K. Khadiev, N. Salikhova, and D. Wu, "On quantum methods for machine learning problems part I: Quantum tools," *Big Data Mining Analytics*, vol. 3, no. 1, pp. 41–55, Mar. 2020, doi: 10.26599/BDMA.2019.9020016.
[38] J. Abhijith, "Quantum algorithm implementations for beginners," *ACM Trans. Quantum Comput.*, vol. 3, no. 4, pp. 1–92, Dec. 2022, doi: 10.1145/3517340.
[39] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th Annu. ACM Symp. Theory Comput. (STOC)*. Philadelphia, PA, USA: ACM Press, 1996, pp. 212–219, doi: 10.1145/237814.237866.

[40] M. Abbasi, F. Cardoso, P. Váz, J. Silva, and P. Martins, "A practical performance benchmark of post-quantum cryptography across heterogeneous computing environments," *Cryptography*, vol. 9, no. 2, p. 32, May 2025, doi: 10.3390/cryptography9020032.
[41] D. J. Bernstein, J. Buchmann, and E. Dahmen, *Post-Quantum Cryptography*. Berlin, Germany: Springer, 2009, doi: 10.1007/978-3-540-88702-7.
[42] D.-T. Dam, T.-H. Tran, V.-P. Hoang, C.-K. Pham, and T.-T. Hoang, "A survey of post-quantum cryptography: Start of a new race," *Cryptography*, vol. 7, no. 3, p. 40, Aug. 2023, doi: 10.3390/cryptography7030040.
[43] T. M. Fernandez-Carames and P. Fraga-Lamas, "Towards post-quantum blockchain: A review on blockchain cryptography resistant to quantum computing attacks," *IEEE Access*, vol. 8, pp. 21091–21116, 2020, doi: 10.1109/ACCESS.2020.2968985.
[44] T. M. Fernandez-Carames, "From pre-quantum to post-quantum IoT security: A survey on quantum-resistant cryptosystems for the Internet of Things," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6457–6480, Jul. 2020, doi: 10.1109/JIOT.2019.2958788.
[45] G. Alagic, D. Apon, D. Cooper, Q. Dang, T. Dang, J. Kelsey, J. Lichtinger, C. Miller, D. Moody, R. Peralta, R. Perlner, A. Robinson, D. S. Tone, and Y. K. Liu, "Status report on the third round of the NIST post-quantum cryptography standardization process," Nat. Inst. Standards Technol., Gaithersburg, MD, USA, NIST Interagency/Internal Rep. (NISTIR) 8413, Jul. 2022, doi: 10.6028/nist.ir.8413.
[46] L. Mariot, S. Picek, and R. Yorgova, "On McEliece-type cryptosystems using self-dual codes with large minimum weight," *IEEE Access*, vol. 11, pp. 43511–43519, 2023, doi: 10.1109/ACCESS.2023.3271767.
[47] D. S. Amirkhanova, M. Iavich, and O. Mamyrbayev, "Lattice-based post-quantum public key encryption scheme using ElGamal's principles," *Cryptography*, vol. 8, no. 3, p. 31, Jul. 2024, doi: 10.3390/cryptography8030031.
[48] D. Swetha and S. K. Mohiddin, "Advancing quantum cryptography algorithms for secure data storage and processing in cloud computing: Enhancing robustness against emerging cyber threats," *Int. J. Adv. Comput. Sci. Appl.*, vol. 15, no. 9, pp. 123–131, 2024, doi: 10.14569/IJACSA.2024.0150964.
[49] T.-H. Nguyen, C.-K. Pham, and T.-T. Hoang, "A high-efficiency modular multiplication digital signal processing for lattice-based post-quantum cryptography," *Cryptography*, vol. 7, no. 4, p. 46, Sep. 2023, doi: 10.3390/cryptography7040046.
[50] M. Iavich and T. Kuchukhidze, "Investigating CRYSTALS-kyber vulnerabilities: Attack analysis and mitigation," *Cryptography*, vol. 8, no. 2, p. 15, Apr. 2024, doi: 10.3390/cryptography8020015.
[51] R. Avanzi, J. Bos, L. Ducas, E. Kiltz, T. Lepoint, V. Lyubashevsky, J. M. Schanck, P. Schwabe, G. Seiler, and D. Stehle, "Algorithm specifications and supporting documentation," NIST Post-Quantum Cryptogr. Project, 2nd Round Submission, Nat. Inst. Standards Technol., Gaithersburg, MD, USA, 2019. [Online]. Available: https://csrc.nist.gov/Projects/Post-Quantum-Cryptography/Round-2-Submissions
[52] L. Ducas, "CRYSTALS-dilithium: A lattice-based digital signature scheme," *TCHES*, vol. 1, pp. 238–268, Feb. 2018, doi: 10.13154/tches.v2018.i1.238-268.
[53] A. Trizna and A. Ozols, "An overview of quantum key distribution protocols," *Inf. Technol. Manage. Sci.*, vol. 21, pp. 37–44, Dec. 2018, doi: 10.7250/itms-2018-0005.
[54] F. Gao, "Practical analysis of discrete variable quantum key distribution," in *Proc. IEEE 2nd Int. Conf. Circuits Syst. (ICCS)*. Chengdu, China: IEEE, Dec. 2020, pp. 140–143, doi: 10.1109/ICCS51219.2020.9336523.
[55] C. H. Bennett and G. Brassard, "Quantum cryptography: Public key distribution and coin tossing," *Theor. Comput. Sci.*, vol. 560, pp. 7–11, Dec. 2014, doi: 10.1016/j.tcs.2014.05.025.
[56] Y. Zhang, Y. Bian, Z. Li, S. Yu, and H. Guo, "Continuous-variable quantum key distribution system: Past, present, and future," *Appl. Phys. Rev.*, vol. 11, no. 1, Mar. 2024, Art. no. 011318, doi: 10.1063/5.0179566.
[57] E. Diamanti and A. Leverrier, "Distributing secret keys with quantum continuous variables: Principle, security and implementations," *Entropy*, vol. 17, no. 9, pp. 6072–6092, Aug. 2015, doi: 10.3390/e17096072.
[58] J. Dong, T. Wang, Z. He, Y. Shi, L. Li, P. Huang, and G. Zeng, "Effective excess noise suppression in continuous-variable quantum key distribution through carrier frequency switching," *Entropy*, vol. 25, no. 9, p. 1286, Aug. 2023, doi: 10.3390/e25091286.
[59] C.-W. Tsai, C.-W. Yang, J. Lin, Y.-C. Chang, and R.-S. Chang, "Quantum key distribution networks: Challenges and future research issues in security," *Appl. Sci.*, vol. 11, no. 9, p. 3767, Apr. 2021, doi: 10.3390/app11093767.



H. A. SHARATH received the M.Tech. degree in computer science and engineering from NMAM Institute of Technology, Nitte, an autonomous institution affiliated with Visvesvaraya Technological University (VTU), Belagavi.

He is currently an Assistant Professor with the Department of Computer Science and Engineering, School of Engineering, Dayananda Sagar University (DSU), Bengaluru. He has more than ten years of teaching experience in engineering colleges and universities. His academic contributions include one international journal article, and two international conference papers. His research interests include cryptography and cybersecurity.

Prof. Sharath has qualified several national and state-level competitive examinations, including the Graduate Aptitude Test in Engineering (GATE), in 2012, the Karnataka State Eligibility Test (KSET), in 2016, and the University Grants Commission National Eligibility Test (UGC-NET), in 2017 and 2019.



JAYAVRINDA VRINDAVANAM received the Ph.D. degree in image processing from NIT, Durgapur, in 2013. She is currently a Professor and the Chairperson of the Department of Computer Science Engineering (Artificial Intelligence (AI) and Machine Learning (ML)). She has more than 20 years of experience in industry, research, and teaching. She has more than 60 publications in international journals, conferences, book chapters, and patents. Her research interests include pattern recognition, artificial intelligence, image processing, machine learning, and computer vision.



SASWATI DANA received the Ph.D. degree in applied mathematics from Indian Institute of Science, Bangalore. She is currently a Researcher with IBM India Research Laboratory, Bangalore. She has more than ten years of experience in Industry and academia. Her current research interests include intersection of AI for programming language and natural language.



S. N. PRASAD (Senior Member, IEEE) was born in Ujire, Dakshina Kannada. He received the Bachelor of Engineering degree from Mangalore University, in 1995, the master's degree from Visvesaraya Technological University, in 2005, and the Ph.D. degree from Jain University, in 2016. He is currently guiding five research scholars in MAHE, five from other universities. Six scholars have already completed their doctoral degree under his guidance from REVA University, Bengaluru. He has 26 years of academic and research experience in technical education. In his career, he has taken several administrative and academic assignments at university level such as, a Professor of electronics and communication engineering, a Special Officer (Evaluation), an Assistant Registrar (Evaluation), an Assistant Director of electronics and communication engineering, and an Assistant Director of placements and corporate relations. He has more than 70 publications in various indexed journals and international conferences. His research interests include artificial intelligence, embedded systems, and signal processing applications. He is a senior member of IETE and ISTE Chapters. He has chaired many international conferences and played a part in organizing an international conference, in 2014 and 2018. He has played a pivotal role in organizing the IEEE flagship Conference-TALE-24at MIT Bengaluru, in December 2024, and was as a Publication Chair in the same.